深蓝学院
shenlanxueyuan.com

多旋翼无人机的运动规划

Motion Planning For Micro Aerial Vehicles

主讲人： 刘思康

2019/09/22



About Me

- Name: Sikang Liu (刘思康)
- Contact: lskwdlskwd@gmail.com
- Experience:
  - Software Engineer at Waymo LLC
- Education:
  - University of Pennsylvania, 2011 – 2018
    - Advisor: Vijay Kumar
  - Shanghai Jiao Tong University, 2008 – 2012
    - Mechanical Engineering



Penn Engineering
GRASP LABORATORY
General Robotics, Automation, Sensing & Perception Lab



Outline

- Introduction
- Preliminary and Technical Review

- Optimization-based Trajectory Planning
- Search-based Trajectory Planning

- Conclusion and Future Work

Outline

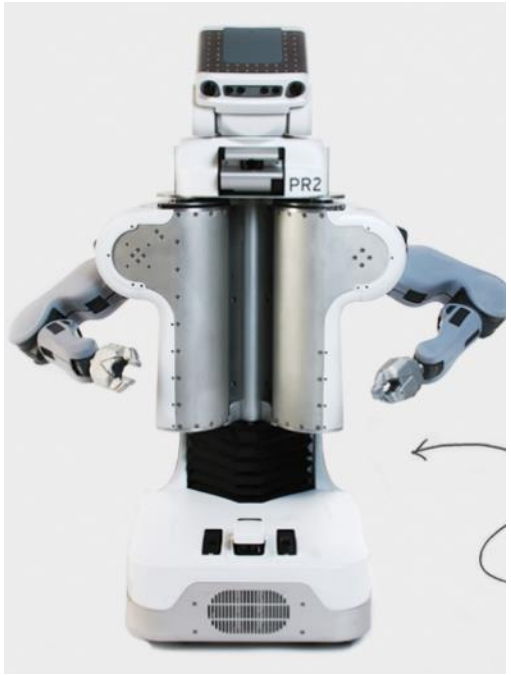
- **Introduction**
- Preliminary and Technical Review
- Optimization-based Trajectory Planning
- Search-based Trajectory Planning
- Conclusion and Future Work

Introduction I: background

- Various mobile platforms:



TurtleBot



PR2



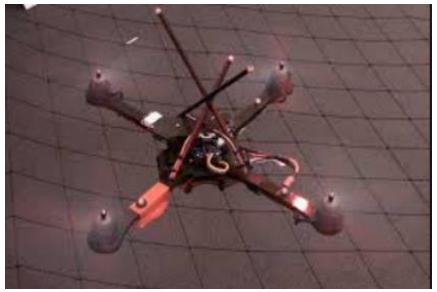
Fixed Wing



Helicopter



Multi-rotor MAV

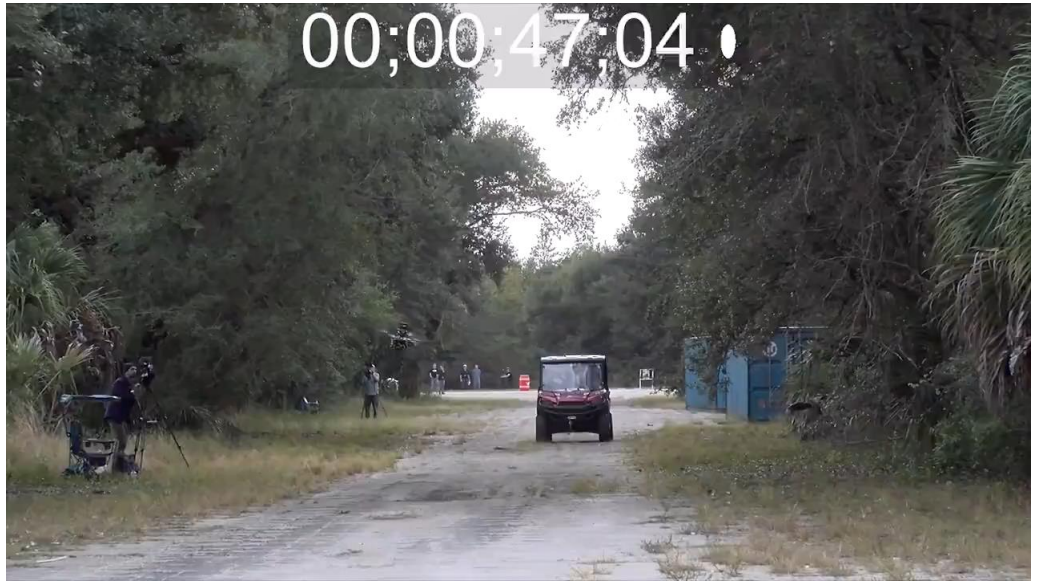


Quadrotor

Micro Aerial Vehicles (MAVs)

Introduction I: background

- MAVs are capable of VTOL and 3D motions



Fly in an unstructured environment



Fly in confined indoor/outdoor environments

Courtesy of DARPA Fast Light-weight Autonomy (FLA) project

Introduction I: background

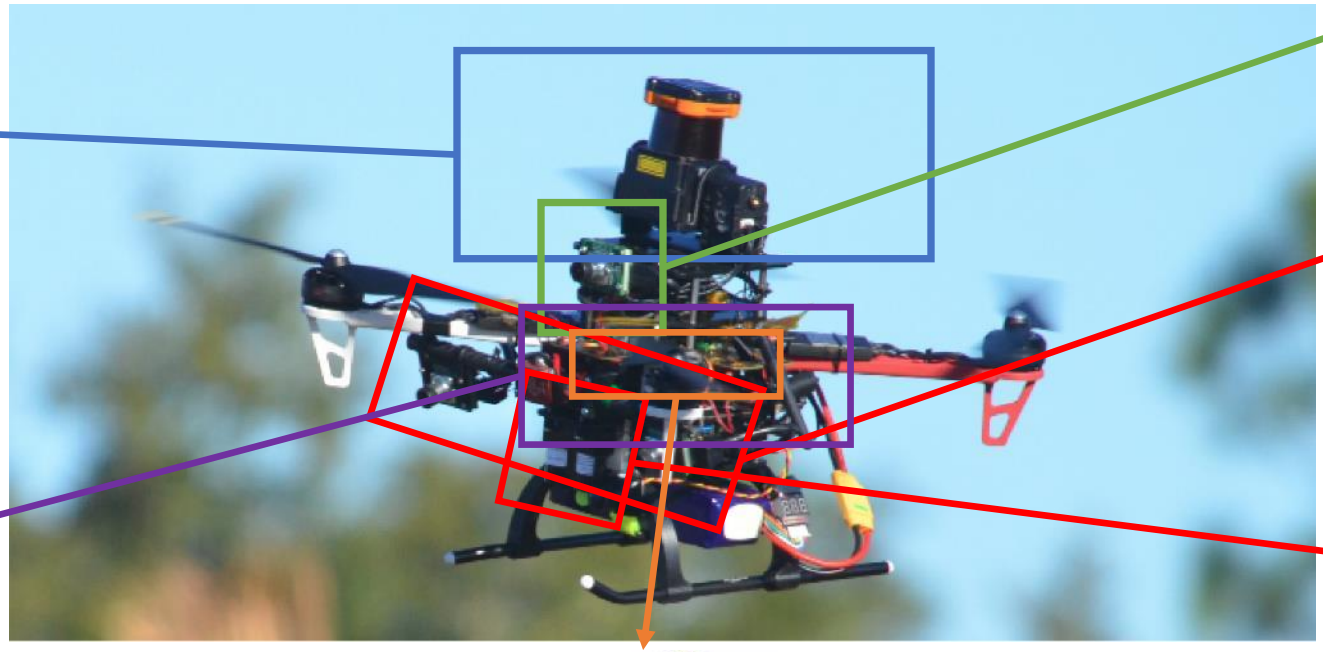
- A fully autonomy MAV



Range Sensor



Computer





Autopilot



RGB Camera



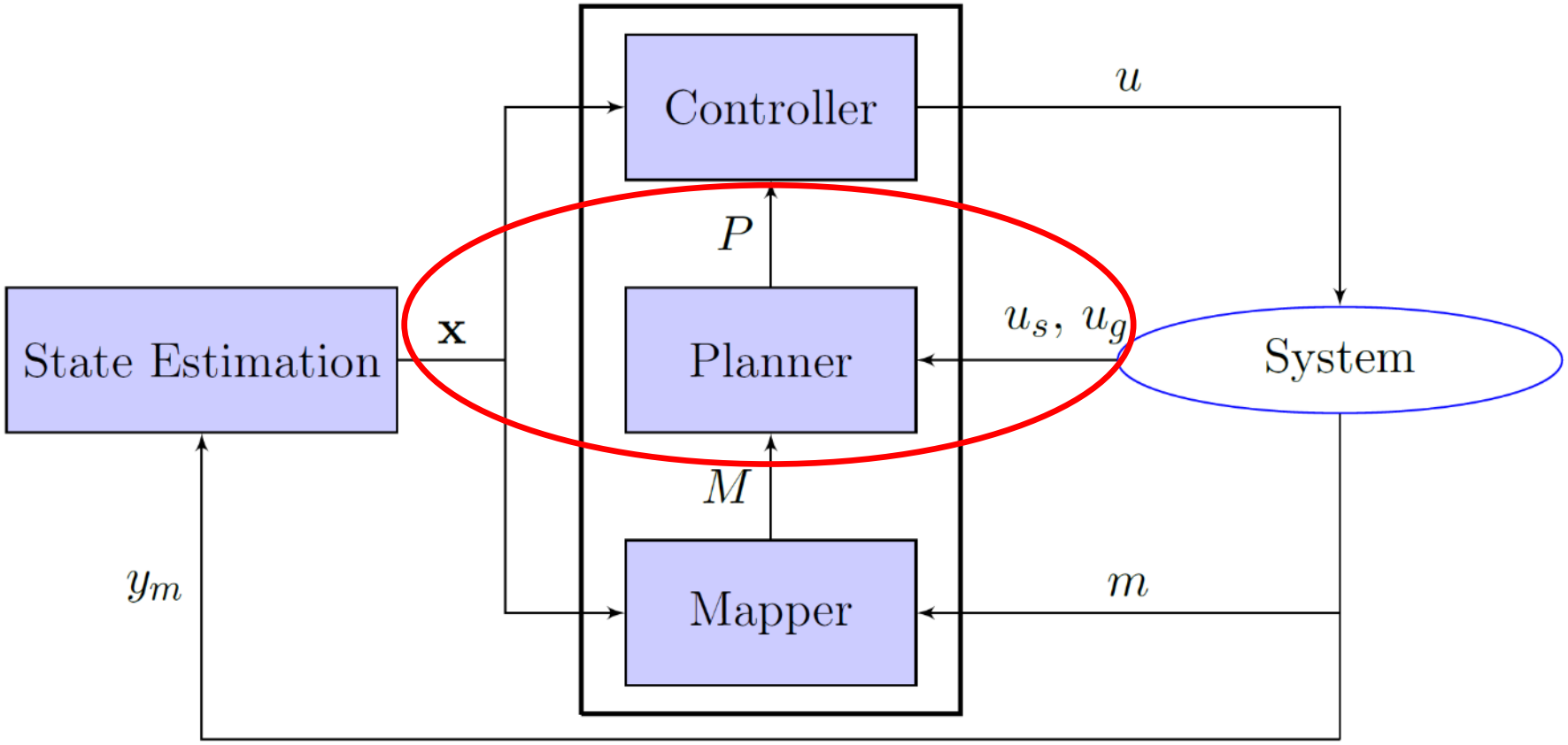
Stereo Camera



Laser Beam

Introduction I: background

- Control Diagram

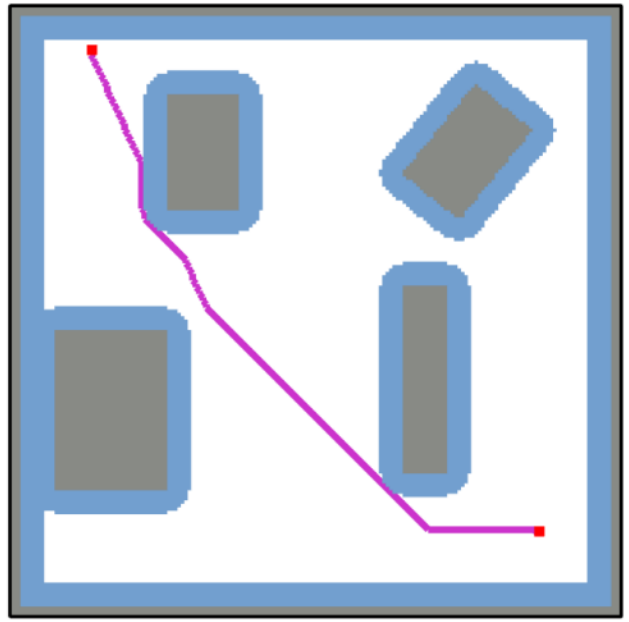


Introduction I: background

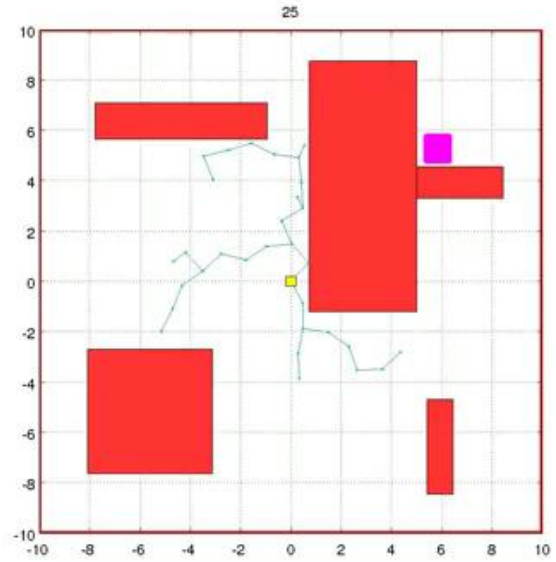
- How to evaluate planner (metrics):
  - Feasibility: whether the planning results are executable or not for MAVs
  - Safety: whether the planning results are collision-free or not
  - Optimality: whether the planning result is optimal or sub-optimal
  - Completeness: if there exists a solution, whether the planner is able to find it
  - Run time: how much time it takes to find the planning result

Introduction II: related work

- Path planning
  - Deterministic: Dijkstra, A*, D*, D* Lite/LPA*, ARA*, and etc.
  - Randomized: PRM, RRT, RRT*, FMT, BIT*, and etc.



Dijkstra

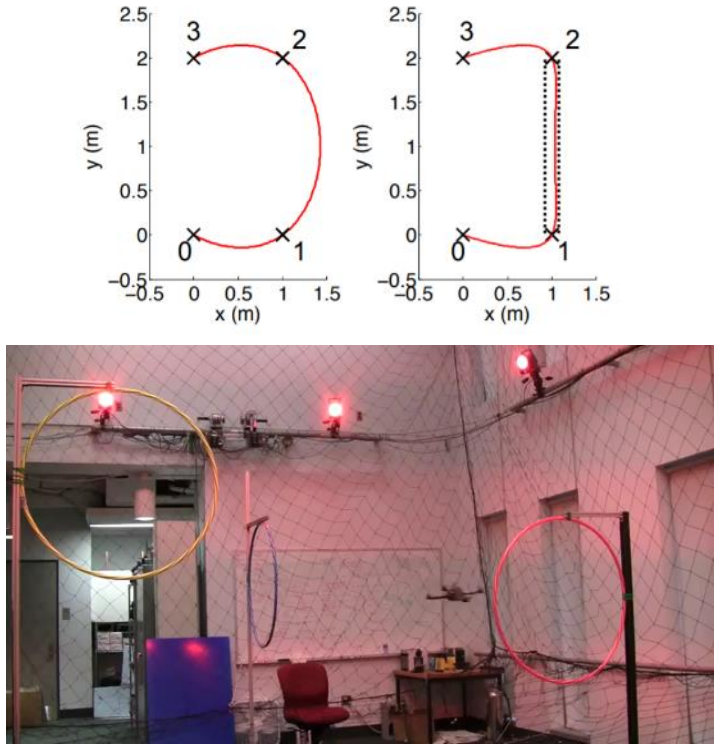


Sertac Karaman, RRT

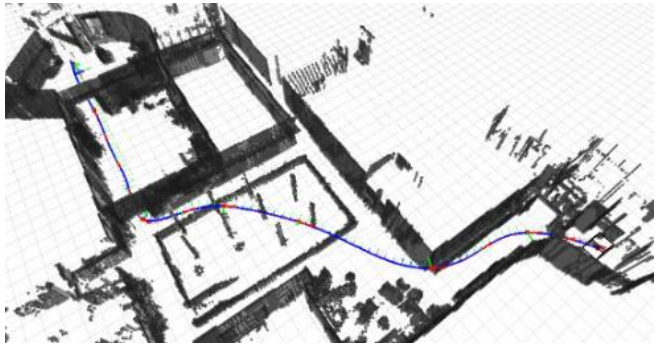
Not dynamically feasible

Introduction II: related work

- Trajectory planning
  - Unconstrained QP: Mellinger 2011 and Richter 2013



D. Mellinger and V. Kumar, Upenn, 2011



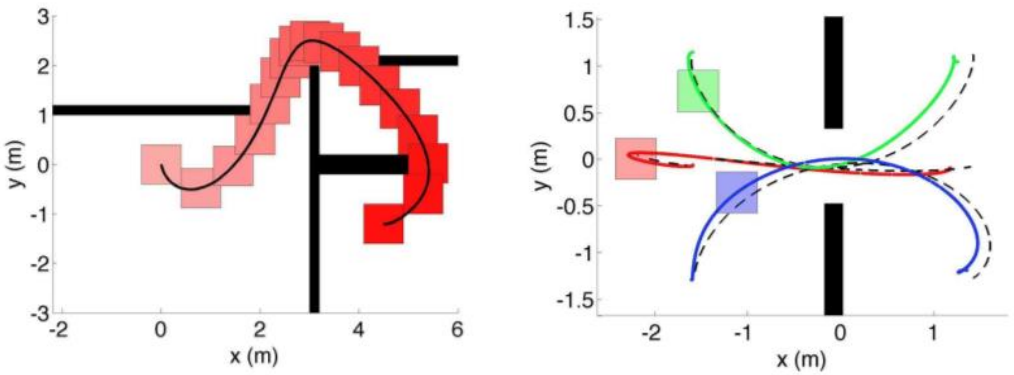


C. Richter and N. Roy, MIT, 2013

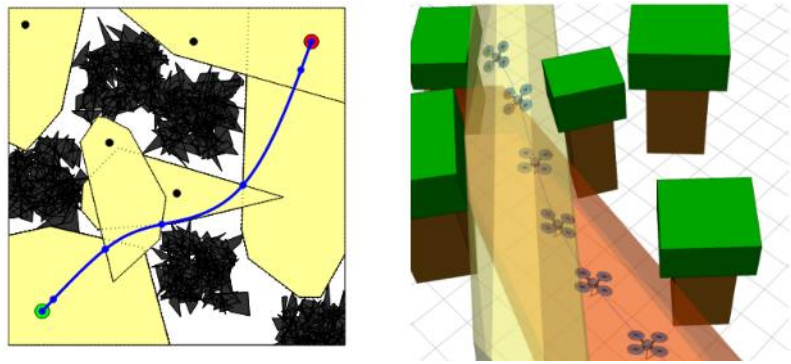
Not handle inequality constraints
Sub-optimal

Introduction II: related work

- Trajectory planning
  - MIP: Mellinger 2012 and Deits 2015



D. Mellinger and V. Kumar, Upenn, 2012

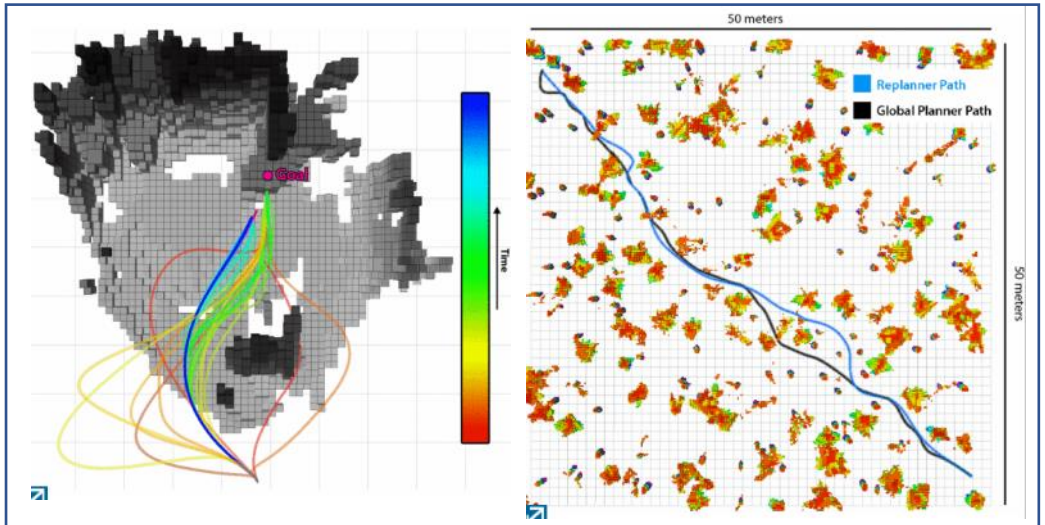


R. Deits and R. Tedrake, MIT, 2015

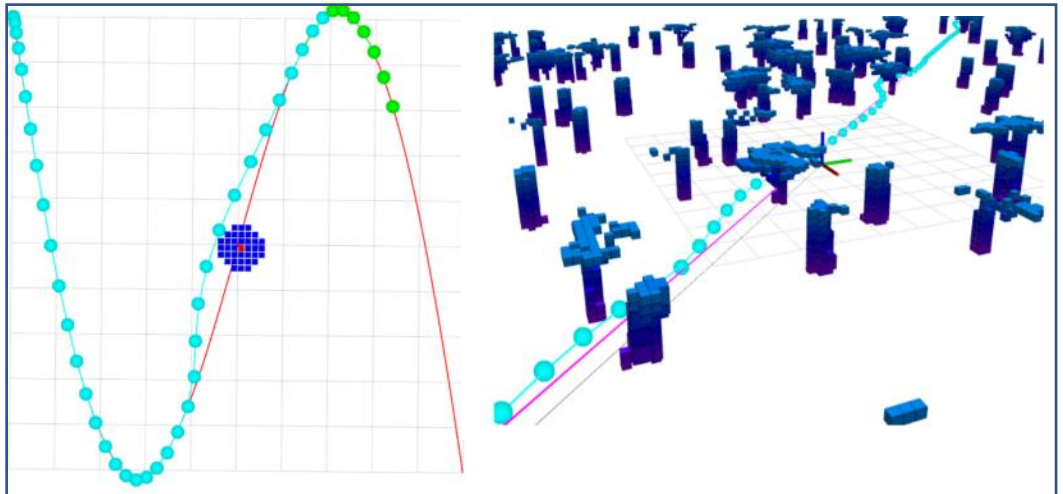
Super slow
Hard to get collision constraints

Introduction II: related work

- Trajectory planning
  - Local planner: Oleynikova 2016 and Usenko 2018



H. Oleynikova and R. Siegwart, ETH, 2016



V. Usenko and D. Cremers, TMU, 2018

Not globally optimal
Not complete

Introduction III: evaluation

- Conclusion of previous work:

Method	***Feasibility***	***Safety***	***Optimality***	***Completeness***	***Run time***
Path Planning	Not feasible	Collision free	Globally optimal	Resolution complete	Very fast

Introduction III: evaluation

- Conclusion of previous work:

Method	***Feasibility***	***Safety***	***Optimality***	***Completeness***	***Run time***
Path Planning	Not feasible	Collision free	Globally optimal	Resolution complete	Very fast
Unconstrained QP	May violate constraints	Not guaranteed	Sub-optimal	Not complete	Very fast

Introduction III: evaluation

- Conclusion of previous work:

Method	***Feasibility***	***Safety***	***Optimality***	***Completeness***	***Run time***
Path Planning	Not feasible	Collision free	Globally optimal	Resolution complete	Very fast
Unconstrained QP	May violate constraints	Not guaranteed	Sub-optimal	Not complete	Very fast
MIP	Dynamically feasible	Collision free	Sub-optimal	Complete	Very slow

Introduction III: evaluation

- Conclusion of previous work:

Method	***Feasibility***	***Safety***	***Optimality***	***Completeness***	***Run time***
Path Planning	Not feasible	Collision free	Globally optimal	Resolution complete	Very fast
Unconstrained QP	May violate constraints	Not guaranteed	Sub-optimal	Not complete	Very fast
MIP	Dynamically feasible	Collision free	Sub-optimal	Complete	Very slow
Short Trajectories	Dynamically feasible	Collision free	Locally optimal	Not complete	Very fast

Outline

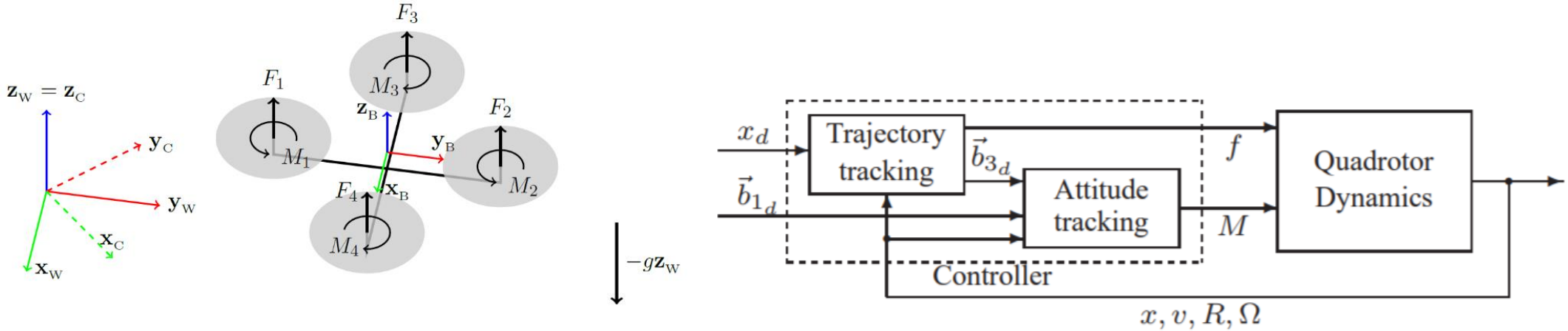
- **Introduction**
- Preliminary and Technical Review
- Optimization-based Trajectory Planning
- Search-based Trajectory Planning
- Conclusion and Future Work

Outline

- Introduction
- **Preliminary and Technical Review**
- Optimization-based Trajectory Planning
- Search-based Trajectory Planning
- Conclusion and Future Work

Preliminary I: model and control

- Differentially flatness



Full state: $\mathbf{x} := [\ x,\ y,\ z,\ \phi,\ \theta,\ \psi,\ \dot{x},\ \dot{y},\ \dot{z},\ p,\ q,\ r\ ]^{\mathsf{T}}.$

Flat output: $\sigma = [\ x,\ y,\ z,\ \psi\ ]^{\mathsf{T}}.$

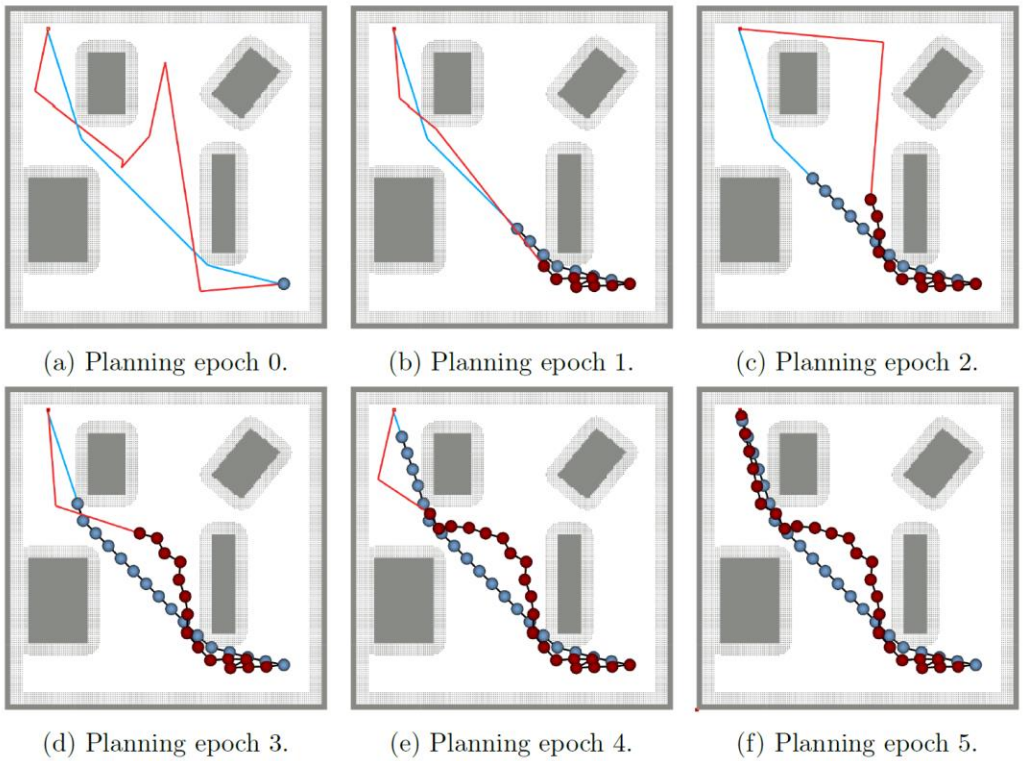
Preliminary II: path planning

- Deterministic vs Randomized:

Dijkstra, A*, D*,
D* Lite/LPA*, ARA*

PRM, RRT, RRT*, FMT,
Informed-RRT*, BIT*

RRT* A*



(a) Planning epoch 0. (b) Planning epoch 1. (c) Planning epoch 2.

(d) Planning epoch 3. (e) Planning epoch 4. (f) Planning epoch 5.

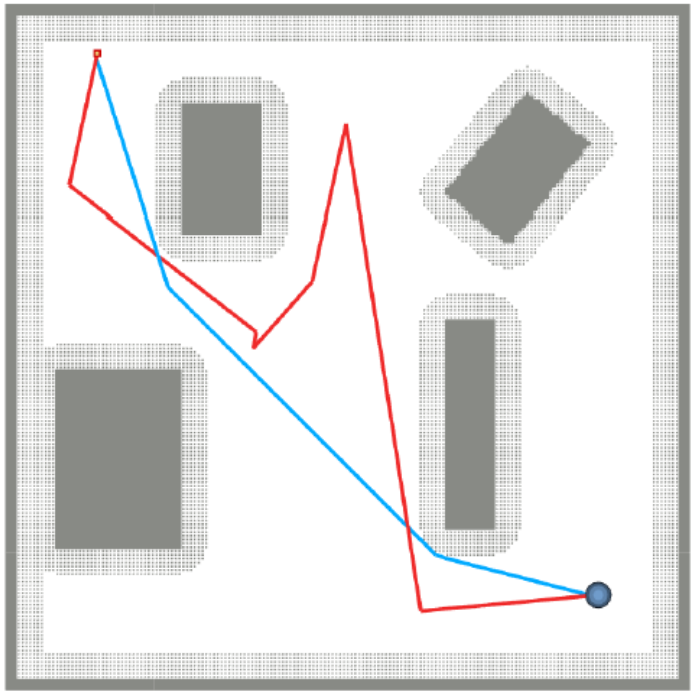
Preliminary II: path planning

- Deterministic vs Randomized:

Dijkstra, A*, D*,
D* Lite/LPA*, ARA*

PRM, RRT, RRT*, FMT,
Informed-RRT*, BIT*

RRT* A*



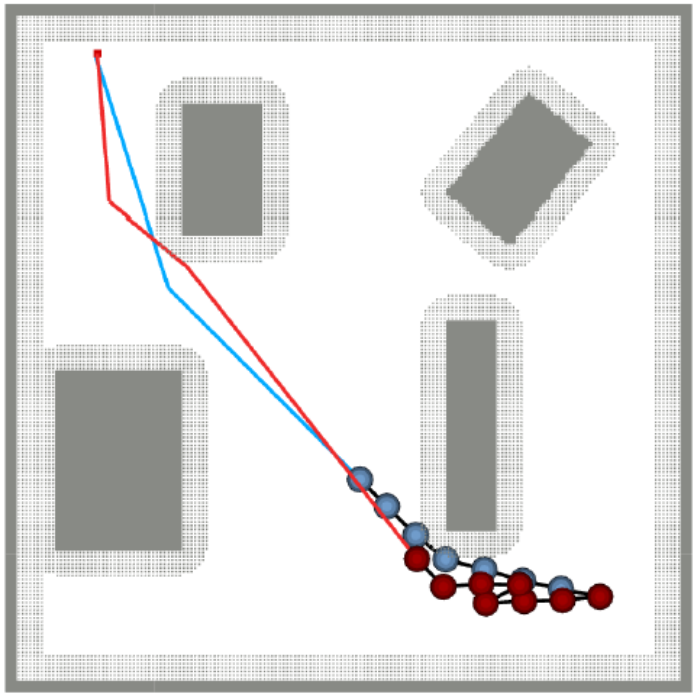
Preliminary II: path planning

- Deterministic vs Randomized:

Dijkstra, A*, D*,
D* Lite/LPA*, ARA*

PRM, RRT, RRT*, FMT,
Informed-RRT*, BIT*

RRT* A*



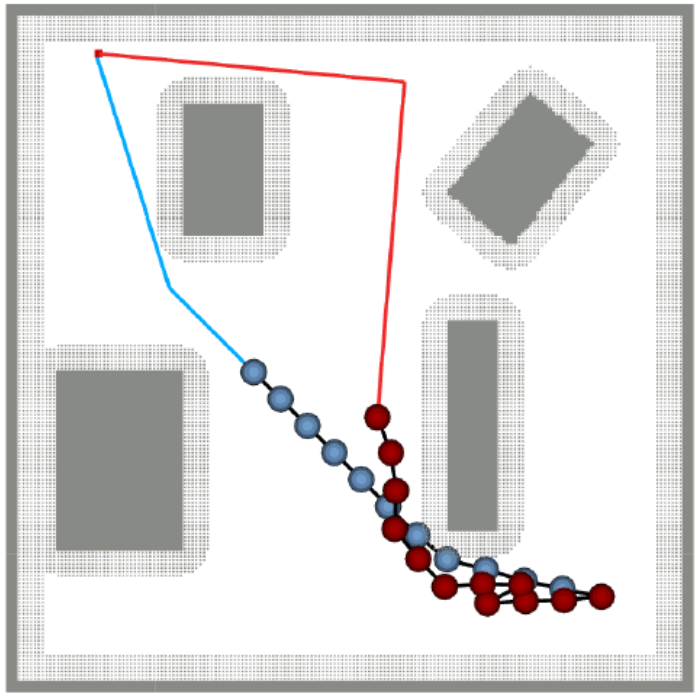
Preliminary II: path planning

- Deterministic vs Randomized:

Dijkstra, A*, D*,
D* Lite/LPA*, ARA*

PRM, RRT, RRT*, FMT,
Informed-RRT*, BIT*

RRT* A*



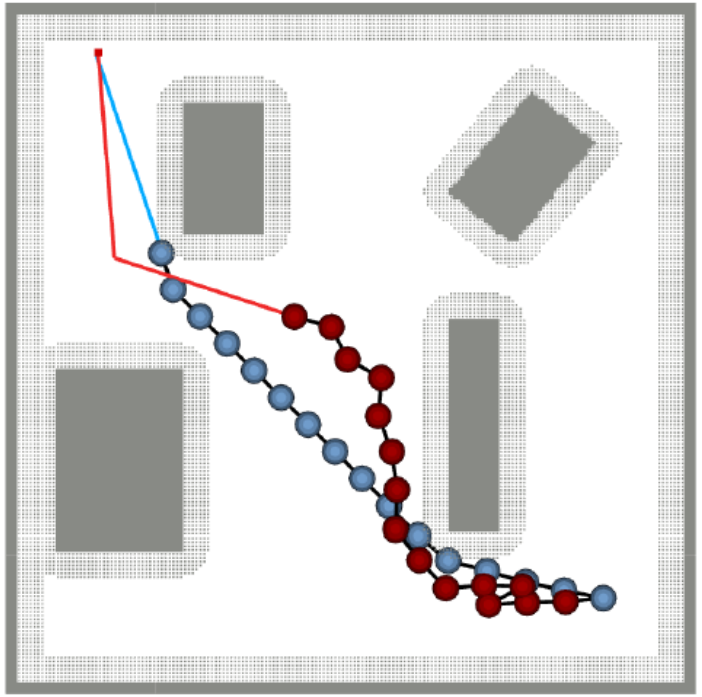
Preliminary II: path planning

- Deterministic vs Randomized:

Dijkstra, A*, D*,
D* Lite/LPA*, ARA*

PRM, RRT, RRT*, FMT,
Informed-RRT*, BIT*

RRT* A*



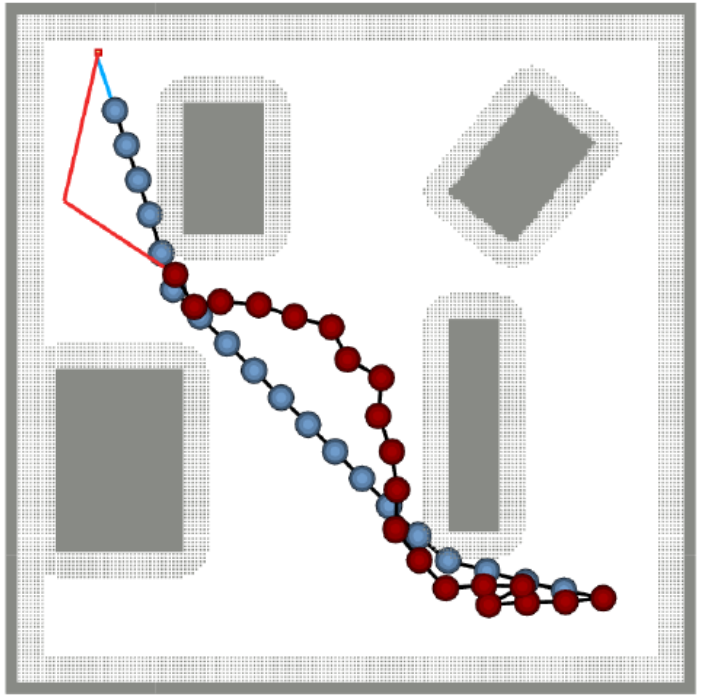
Preliminary II: path planning

- Deterministic vs Randomized:

Dijkstra, A*, D*,
D* Lite/LPA*, ARA*

PRM, RRT, RRT*, FMT,
Informed-RRT*, BIT*

RRT* A*



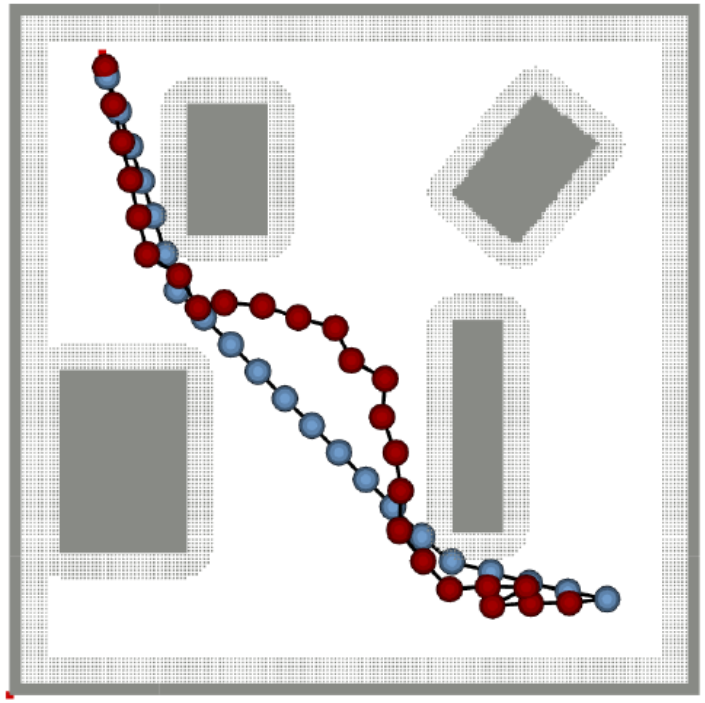
Preliminary II: path planning

- Deterministic vs Randomized:

Dijkstra, A*, D*,
D* Lite/LPA*, ARA*

PRM, RRT, RRT*, FMT,
Informed-RRT*, BIT*

RRT* A*

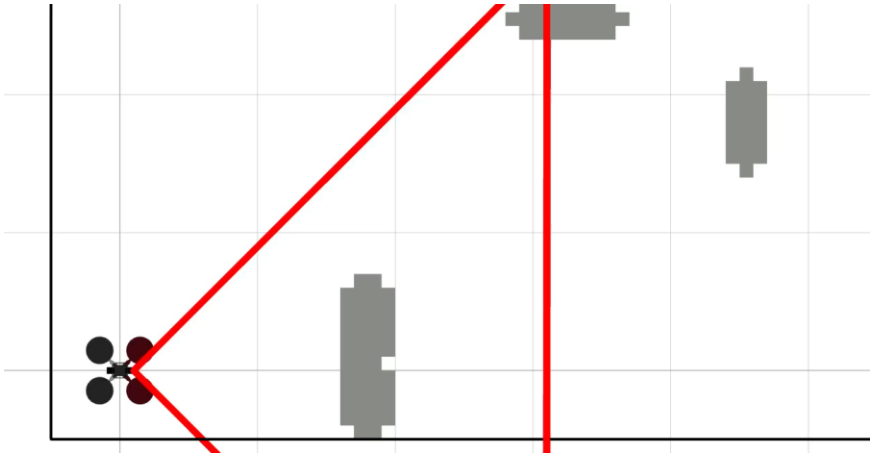


Preliminary III: control paradigm

- Convert the spatial path into flat outputs:
  - reference governor method

$$p_d(t+dt) = p_d(t) + v_d(t)dt,$$

$$\psi_d(t+dt) = \psi_d(t) + w_d(t)dt.$$



- Drawbacks:
  - Not respect dynamics constraints
  - Not guarantee safety
  - Only works for low speed

Outline

- Introduction
- **Preliminary and Technical Review**
- Optimization-based Trajectory Planning
- Search-based Trajectory Planning
- Conclusion and Future Work

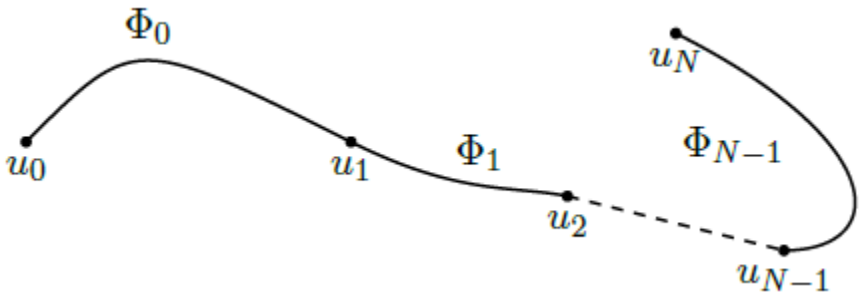
Outline

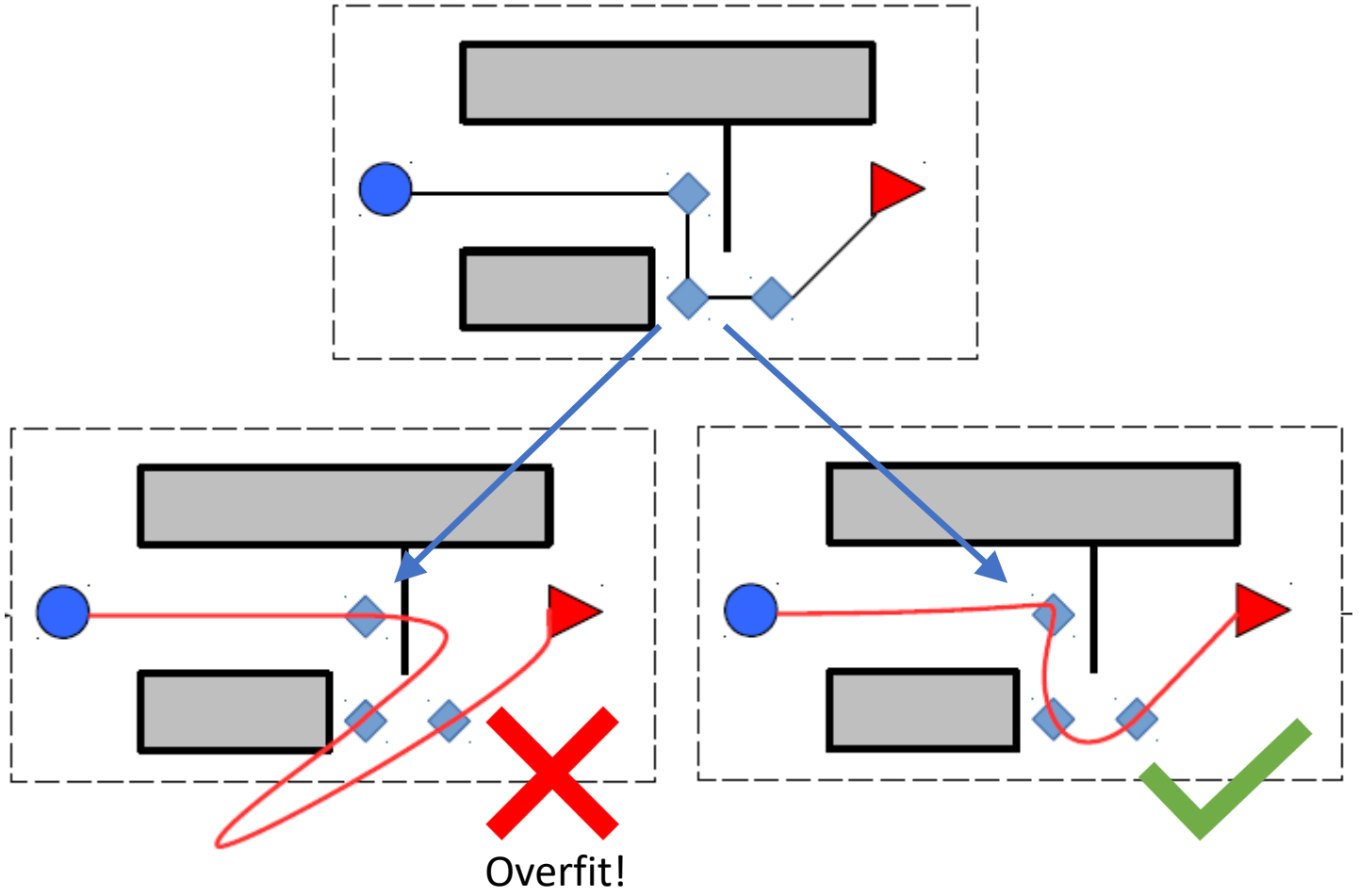
- Introduction
- Preliminary and Technical Review
- **Optimization-based Trajectory Planning**
- Search-based Trajectory Planning
- Conclusion and Future Work

Optimization-based Trajectory Planning

- Piece-wise Polynomial:

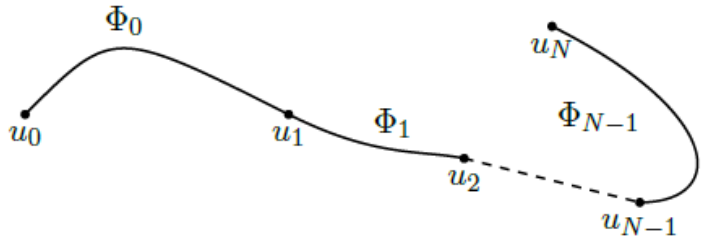
$$\Phi(t) = \begin{cases} \Phi_0(t - t_0), & t_0 \le t < t_1, \\ \Phi_1(t - t_1), & t_1 \le t < t_2, \\ \vdots & \\ \Phi_{N-1}(t - t_{N-1}), & t_{N-1} \le t \le t_N. \end{cases}$$





Optimization-based Trajectory Planning

- Unconstrained Quadratic Programming (QP):
  - Cost function in terms of coefficients:



$$J^q(p) = \int_0^{\Delta t} \|p^{(q)}(t)\|^2 dt = \mathbf{a}^\mathsf{T} Q^q(\Delta t)\mathbf{a}$$

  - Waypoint Constraints:

$$A(0)\cdot\mathbf{a} = b(0),\ \ A(\Delta t)\cdot\mathbf{a} = b(\Delta t).$$

$$A = \begin{bmatrix} A_0(0) & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & A_0(\Delta t_0) & \cdots & \mathbf{0} & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & A_{N-1}(0) & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & A_{N-1}(\Delta t_{N-1}) \end{bmatrix}.$$

  - Problem re-formulation:

$$\begin{aligned} &\arg\min_{\mathbf{a}}\ \mathbf{a}^\mathsf{T} Q\mathbf{a} \\ &s.t\ A\mathbf{a} = b \end{aligned}$$

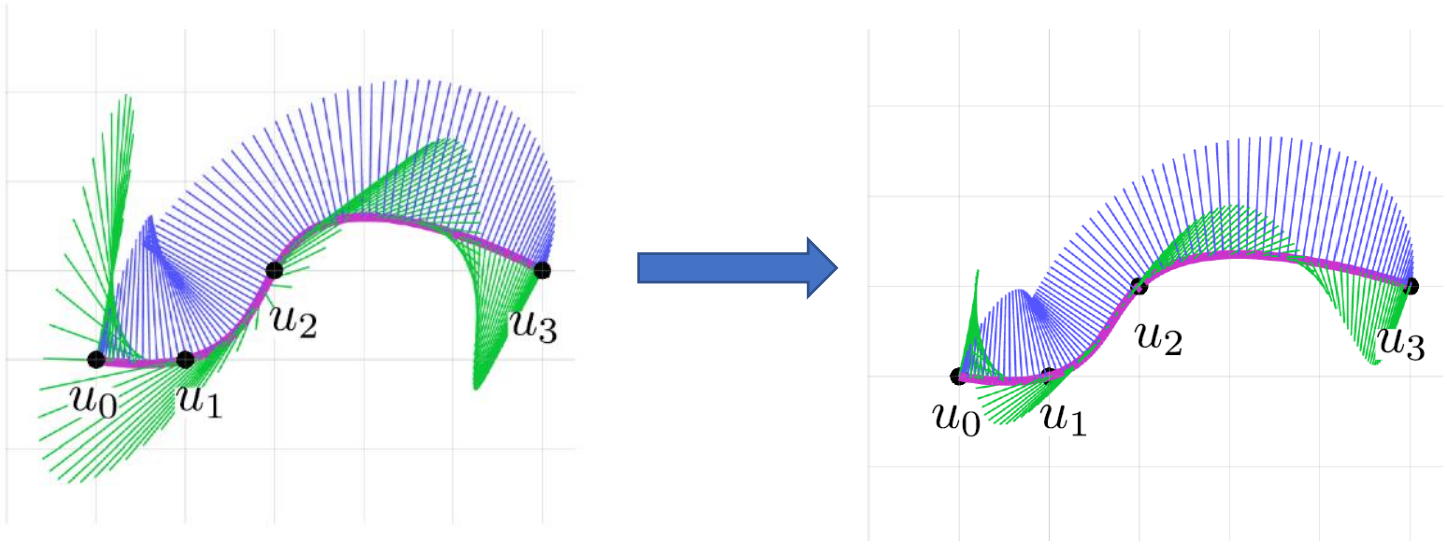
Closed-form!

$$\mathbf{a}^* = A^{-1}M\begin{bmatrix} D_F \\ D_P^* \end{bmatrix} = A^{-1}M\begin{bmatrix} I \\ -R_{PP}^{-1}R_{FP}^T \end{bmatrix} D_F.$$

Optimization-based Trajectory Planning

- Unconstrained QP:
  - Dynamics constraints: $v_{\max} \leq \bar{v}_{\max},\ a_{\max} \leq \bar{a}_{\max},\ j_{\max} \leq \bar{j}_{\max}.$
  - Solution: Scale time allocation to satisfy dynamics constraints.

$$\Delta t' = \max\{1, (\frac{v_{\max}}{\bar{v}_{\max}}), (\frac{a_{\max}}{\bar{a}_{\max}})^{\frac{1}{2}}, (\frac{j_{\max}}{\bar{j}_{\max}})^{\frac{1}{3}}\}\Delta t.$$



Original

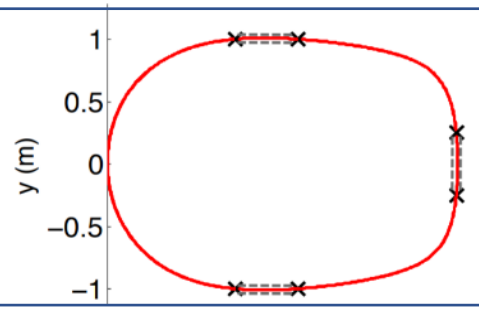
Re-optimized

Optimization-based Trajectory Planning

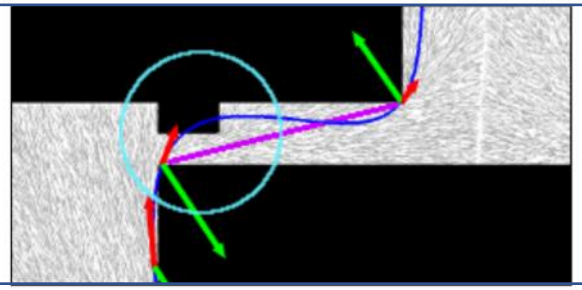
- Unconstrained QP:
  - Collision constraints: $x(t) \in \mathcal{X}^{free}$
  - Existing solutions: add intermediate waypoints

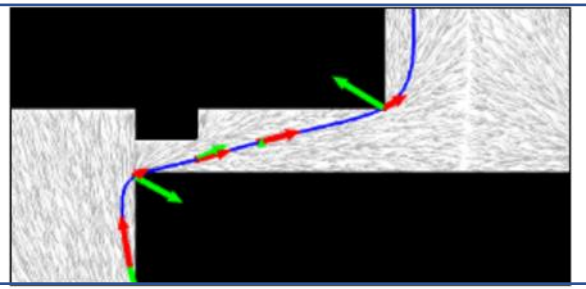
Mellinger 2011:

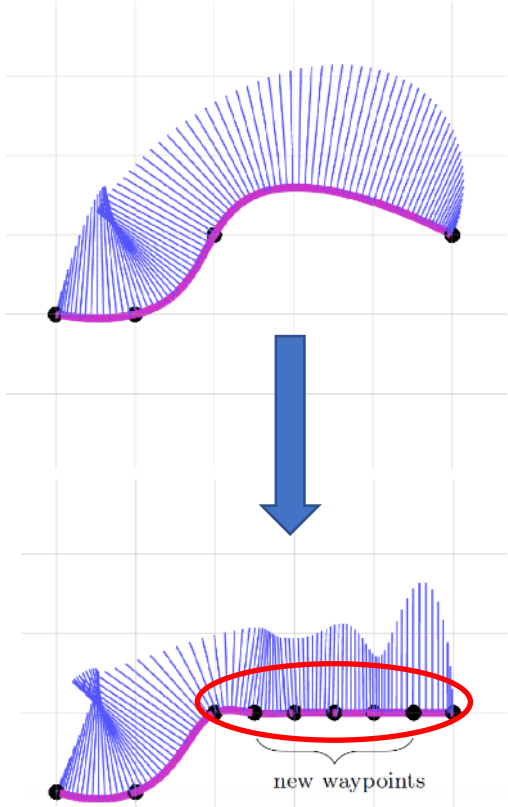




Richter 2013:



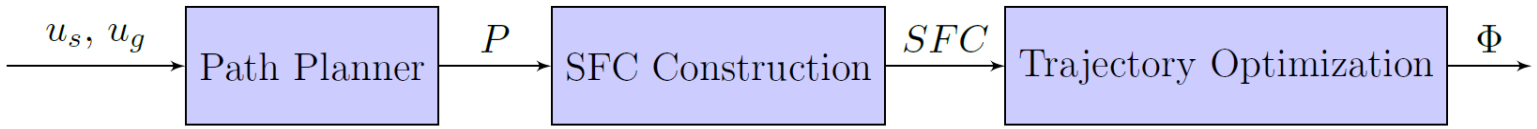




Velocity profile is not flat!

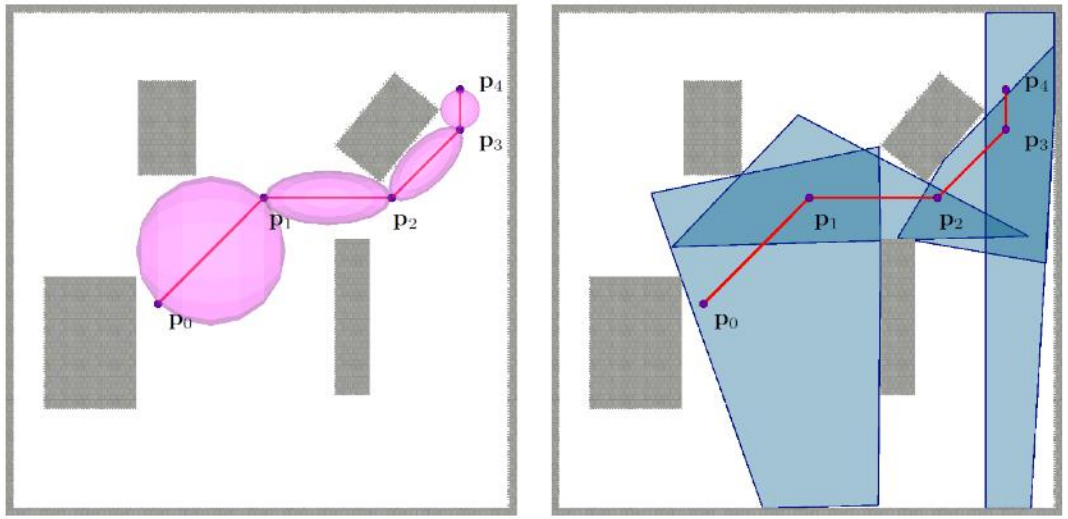
Optimization-based Trajectory Planning

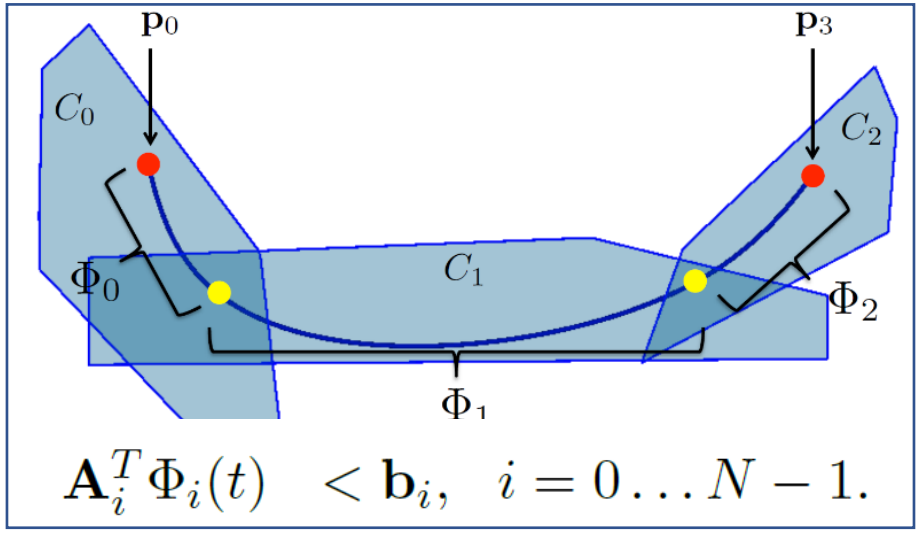
- Using Quadratic Programming (QP) :
  - Solve QP within a Safe Flight Corridor (SFC) [Liu, ICRA 2017]



$$\|v\|_\infty \le v_{max},$$
$$\|a\|_\infty \le a_{max},$$
$$\vdots$$

Dynamics constraints

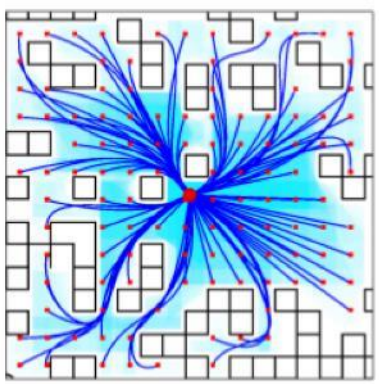




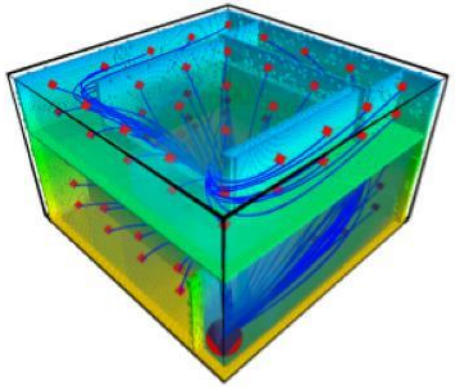
Collision constraints

Optimization-based Trajectory Planning

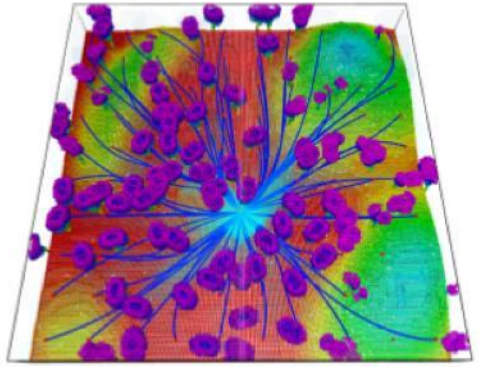
- Run time analysis:



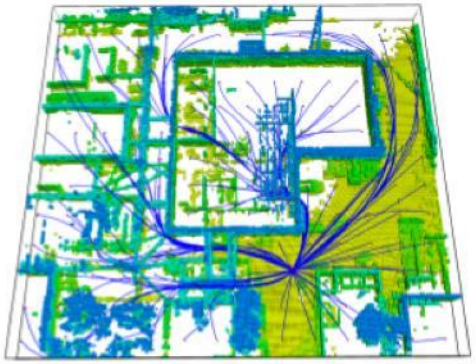
(a) Random Blocks.



(b) Multiple Floors.



(c) The Forest.

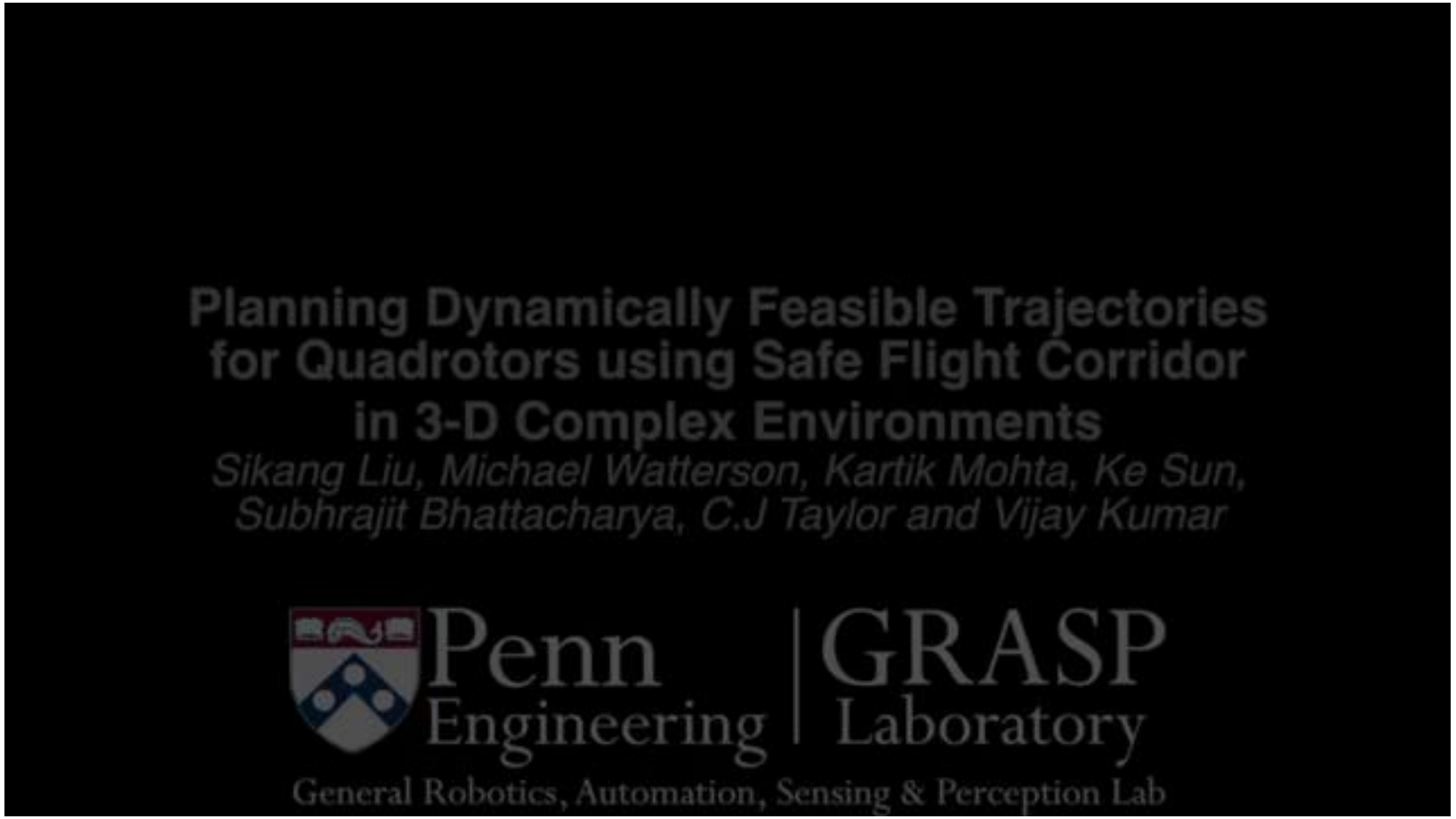


(d) Outdoor Buildings.

Map	Size	# of Cells	# of Trajs	Time (s)	Path Planning		Convex Decomp	Traj Opt	Replan (JPS)
					A*	JPS			
Random Blocks	$40 \times 40 \times 1$	1.4×10^6	130	Avg	0.57	0.034	0.0021	0.028	0.065
				Std	1.26	0.034	0.0028	0.022	0.051
				Max	9.98	0.19	0.020	0.099	0.27
Multiple Floors	$10 \times 10 \times 6$	5.9×10^5	147	Avg	6.12	0.039	0.0064	0.082	0.13
				Std	15.77	0.046	0.0038	0.041	0.081
				Max	84.56	0.22	0.021	0.23	0.45
The Forest	$50 \times 50 \times 6$	1.8×10^6	89	Avg	0.65	0.033	0.0039	0.055	0.094
				Std	1.57	0.044	0.0024	0.031	0.068
				Max	7.78	0.20	0.010	0.12	0.30
Outdoor Buildings	$100 \times 110 \times 7$	6.2×10^5	127	Avg	0.54	0.028	0.0066	0.099	0.14
				Std	1.46	0.045	0.0053	0.064	0.10
				Max	10.96	0.27	0.027	0.24	0.47

~100ms

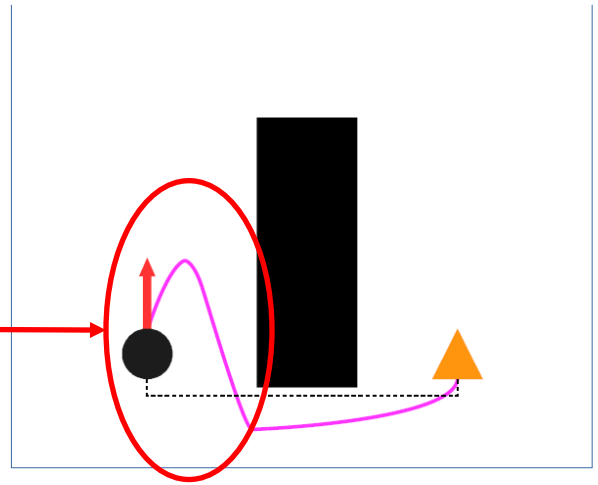
Optimization-based Trajectory Planning

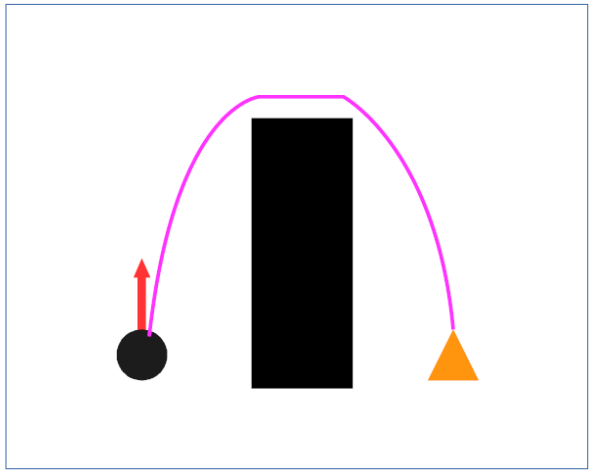


Optimization-based Trajectory Planning

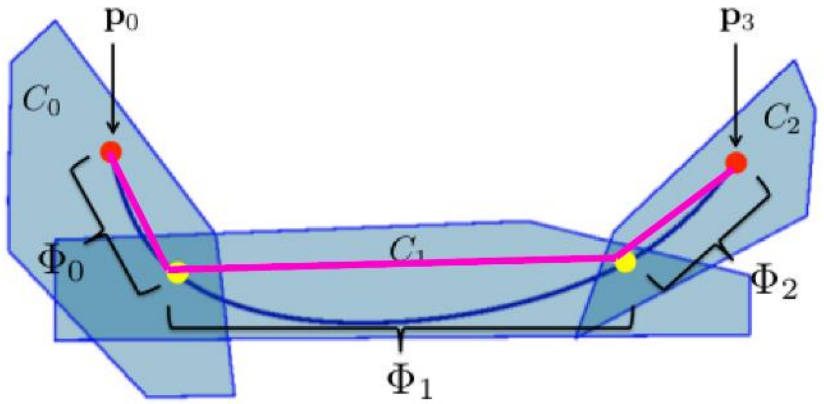
- Searching homology class:

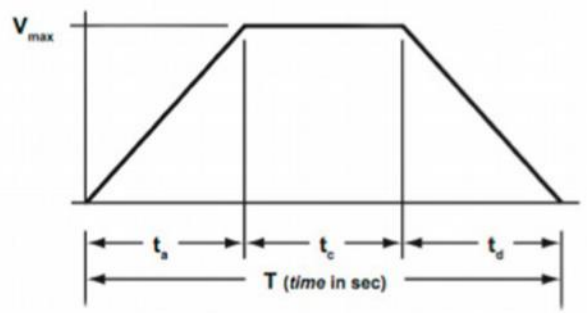
Too aggressive !





- Time allocation:





Optimization-based Trajectory Planning

- Conclusion

Method	***Feasibility***	***Safety***	***Optimality***	***Completeness***	***Run time***
SFC	Dynamically feasible	Collision free	Sub-optimal	Complete*	Fast

~100ms

* means conditionally complete

Outline

- Introduction
- Preliminary and Technical Review
- **Optimization-based Trajectory Planning**
- Search-based Trajectory Planning
- Conclusion and Future Work

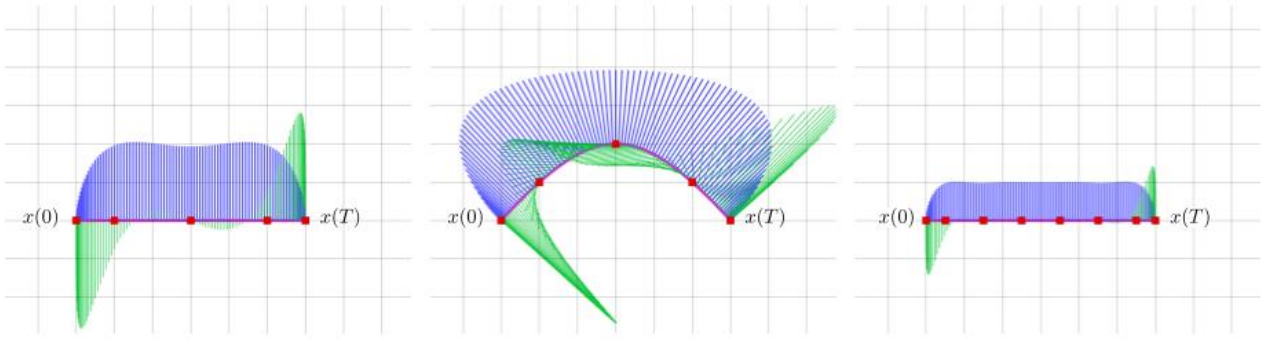
Outline

- Introduction
- Preliminary and Technical Review
- Optimization-based Trajectory Planning
- **Search-based Trajectory Planning**
- Conclusion and Future Work

Search-based Trajectory Planning

- Problem revisit:
  - global optimality
  - time allocation

$$J(D) := \int_0^T \|u(t)\|^2 \, dt = \int_0^T \left\| p_D^{(n)}(t) \right\|^2 dt$$



(a) $T = 4, J = 19.$ (b) $T = 4, J = 48.$ (c) $T = 7, J = 5.$

$$\arg\min_{D,T} \; J(D) + \rho T$$

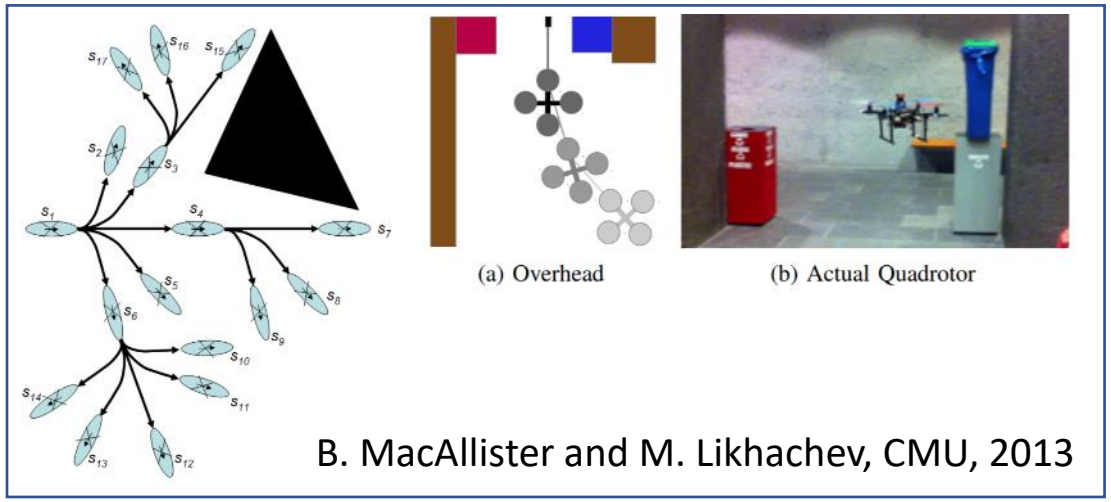
$$s.t. \quad \dot{x}(t) = Ax(t) + Bu(t), \quad \forall\, t \in [0,\ T]$$

$$x(0) = x_0, \quad x(T) \in \mathcal{X}^{goal}$$

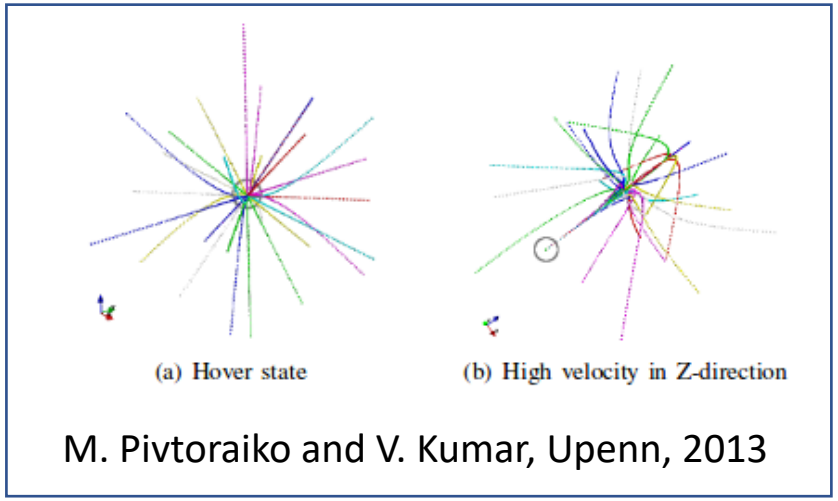
$$x(t) \in \mathcal{X}^{free}, \quad u(t) \in \mathcal{U}, \quad \forall\, t \in [0,\ T]$$

Search-based Trajectory Planning

- Motion primitives:
  - Existing work: sampling in state space



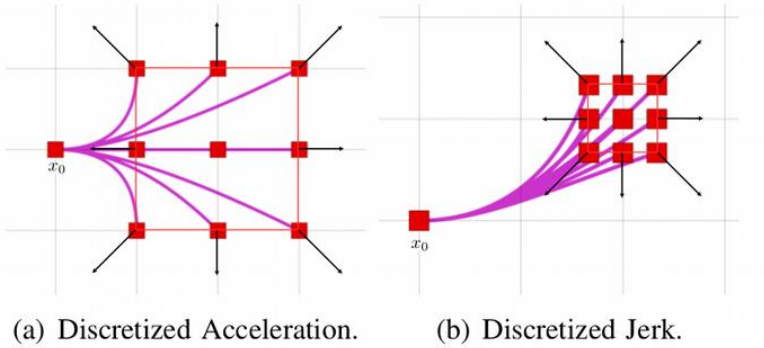
B. MacAllister and M. Likhachev, CMU, 2013



M. Pivtoraiko and V. Kumar, Upenn, 2013

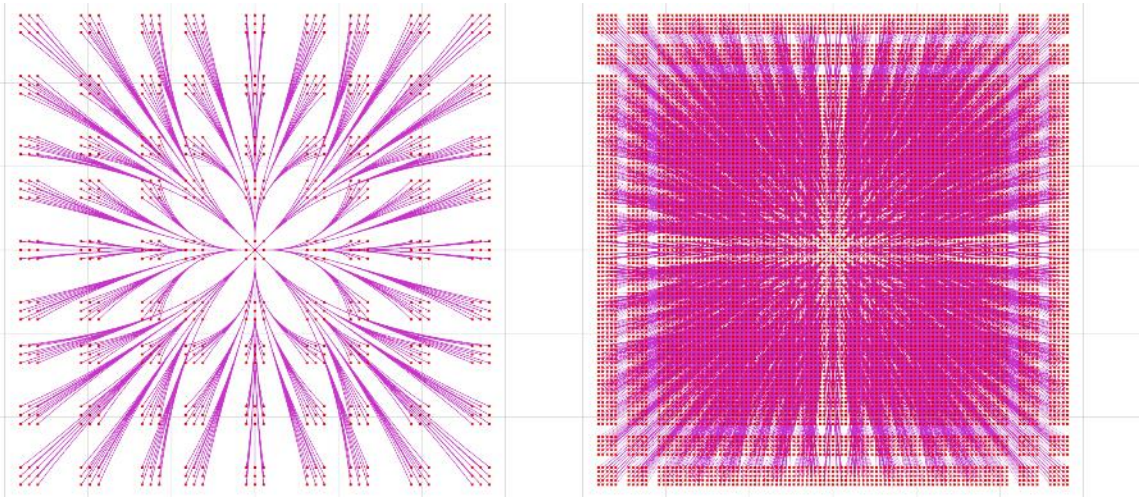
  - Our method: sampling in control space
    - Online primitive generation

$$p_D(t) = u_m \frac{t^q}{q!} + \ldots + a_0 \frac{t^2}{2} + v_0 t + p_0.$$

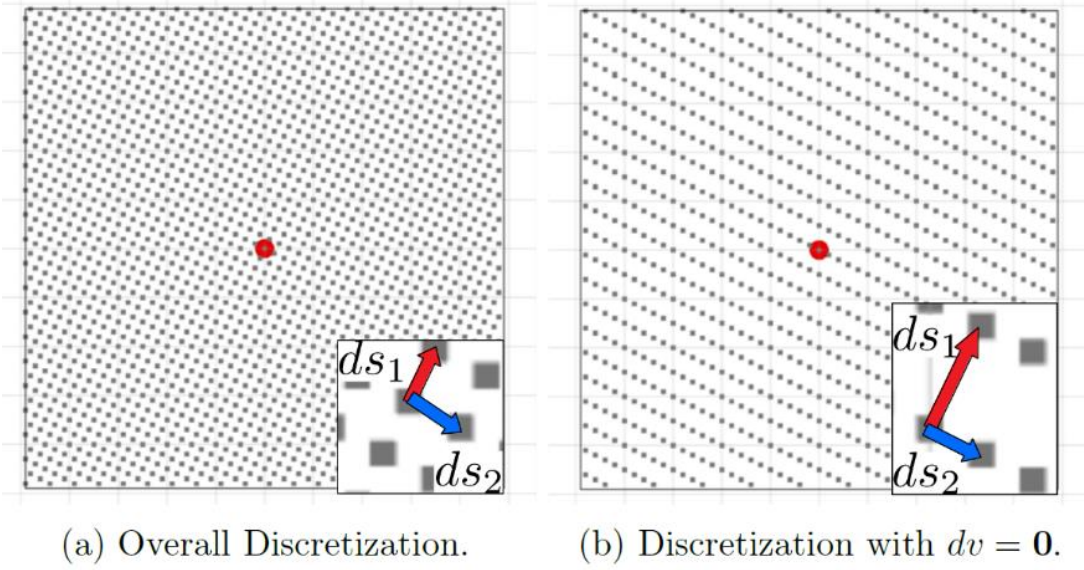


Search-based Trajectory Planning

- Induced space discretization:



Example induced graph from a static state



(a) Overall Discretization. (b) Discretization with $dv = \mathbf{0}$.

Example induced graph from a non-static state

Search-based Trajectory Planning

- Deterministic shortest trajectory problem:

$$\min_{N, u_{0:N-1}} \left(\sum_{k=0}^{N-1} \|u_k\|^2 + \rho N \right) \tau$$

$$s.t.\ x_k(\tilde{t}) = F(\tilde{t}) s_k + H(\tilde{t}) u_k \subset \mathcal{X}^{free}, \quad \tilde{t} \in [0, \tau]$$

$$x_k(\tilde{t}) \subset \mathcal{X}^{free} \quad \forall\, k \in \{0, \ldots, N-1\},\ \tilde{t} \in [0, \tau]$$

$$s_{k+1} = x_k(\tau), \quad \forall\, k \in \{0, \ldots, N-1\}$$

$$s_0 = x_0, \quad s_N \in \mathcal{X}^{goal}$$

$$u_k \in \mathcal{U}_M, \quad \forall\, k \in \{0, \ldots, N-1\}$$

Search-based Trajectory Planning

- Solution:
  - A* with proper heuristic

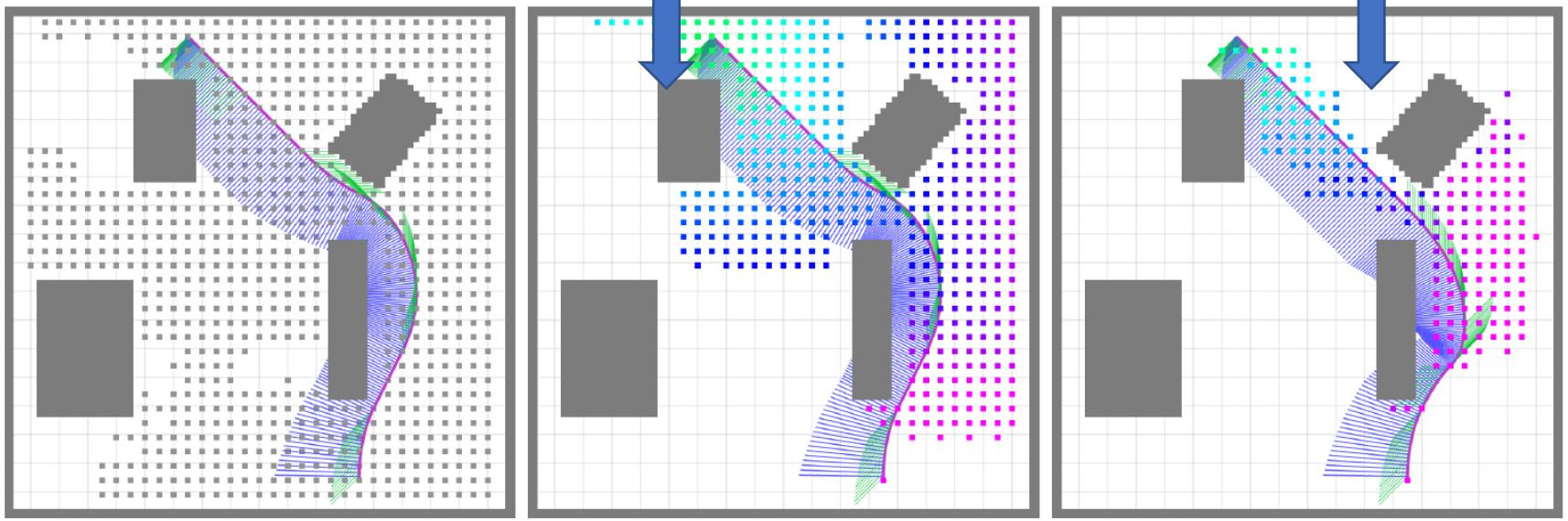
1. Minimum time heuristic:

$$h_1 = \rho \max(\bar{T}_v, \bar{T}_a)$$

$$\bar{T}_v = \|p_f - p_0\|_\infty / v_{max}$$

2. Linear quadratic minimum time:

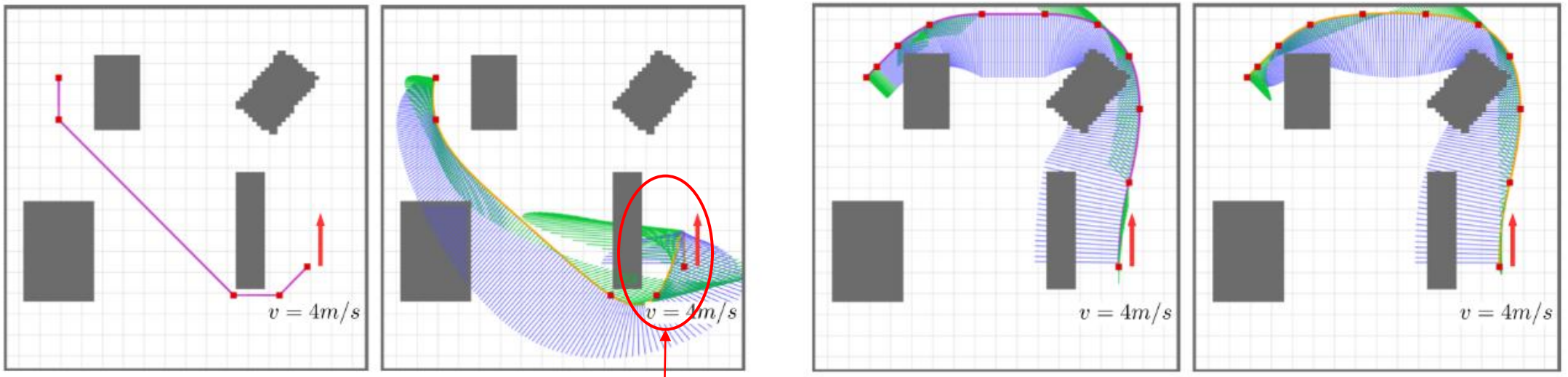
$$\min_{D,T} \; J(D) + \rho T$$
$$\text{s.t. } \dot{x}(t) = Ax(t) + Bu(t), \quad \forall t \in [0, T]$$
$$x(0) = x_0, \quad x(T) \in \mathcal{X}^{goal}$$
$$T \geq \bar{T}$$



(a) Dijkstra. (b) A^* with h_1. (c) A^* with h_2.

Search-based Trajectory Planning

- Comparing to optimization-based method:



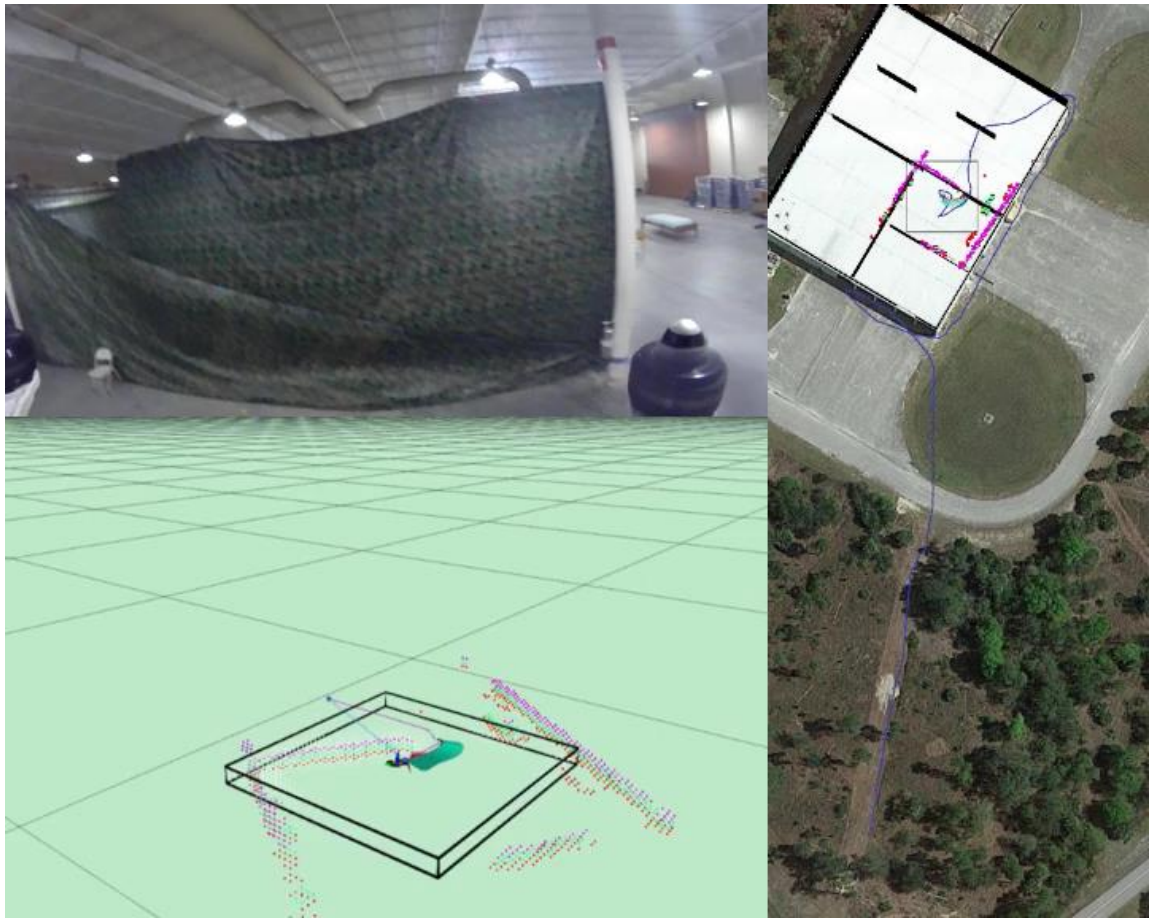
(a) $T = 8.5$. (b) $T = 8.5, J = 296.6$. (c) $T = 10, J = 14.0$. (d) $T = 10, J = 21.3$.

Optimization-based Approach

Search-based Approach

Too aggressive !

Search-based Trajectory Planning

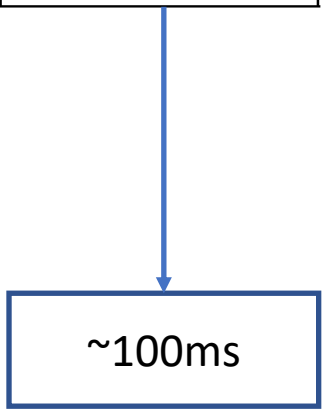


FLA Phase1, 2017

Search-based Trajectory Planning

- Conclusion:

Method	***Feasibility***	***Safety***	***Optimality***	***Completeness***	***Run time***
SMP	Dynamically feasible	Collision free	Resolution optimal	Globally complete	Fast



Search-based Trajectory Planning: Extensions

- List of contents:
  - Trajectory planning with motion uncertainty
  - Trajectory planning with limited field-of-view (FOV)
  - Trajectory planning in SE(3)

- General problem formulation:

Cost →

$$\arg\min_{\Phi} \; J_q(\Phi) + \rho_T T$$

$$s.t. \quad \dot{x}(t) = Ax(t) + Bu(t),$$

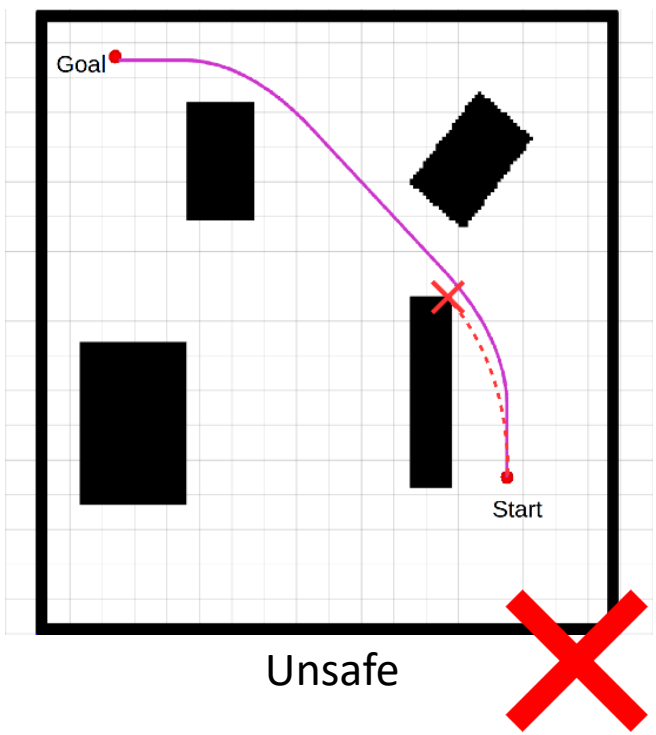
Constraints →

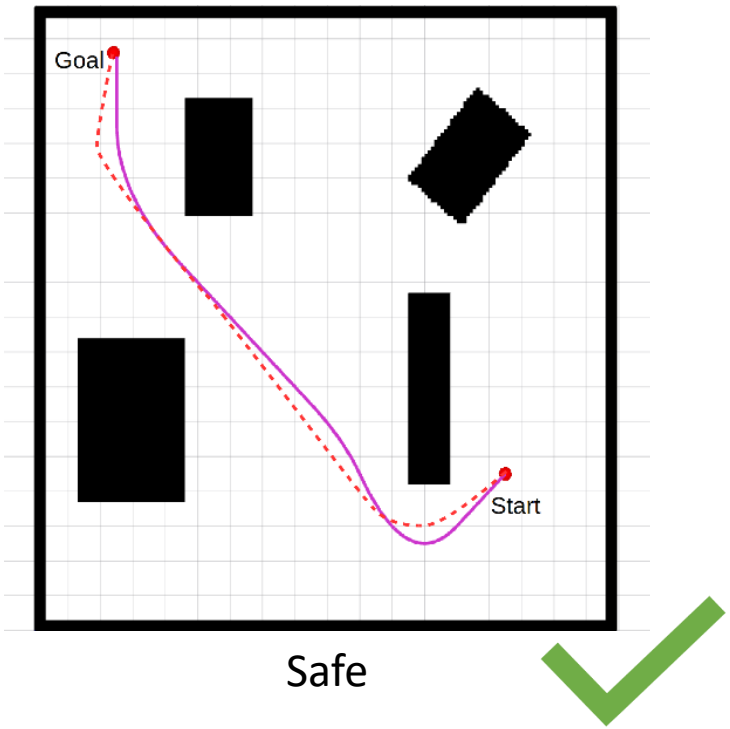
$$x(0) = x_0, \quad x(T) \in \mathcal{X}^{goal},$$

$$x(t) \in \mathcal{X}^{free}, \quad u(t) \in \mathcal{U}.$$

Extensions I: motion uncertainty

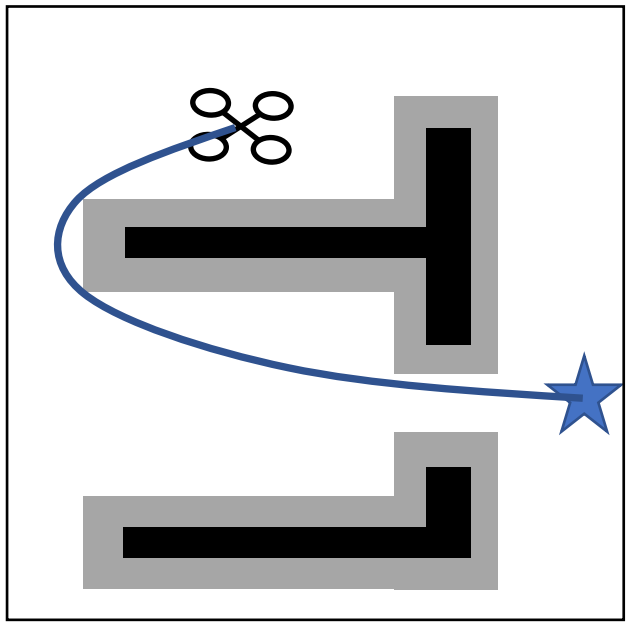
- Motivations:
  - High control authority is impractical
  - Real-world environmental factors: wind, air-drag, wall effect and etc



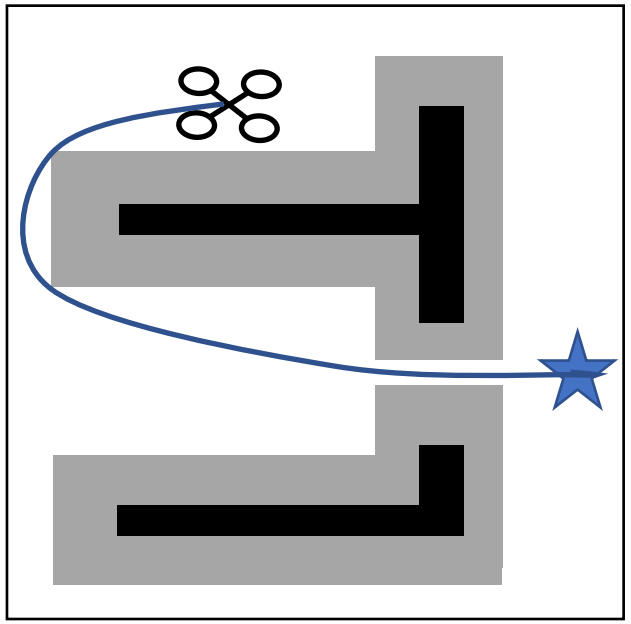


Extensions I: motion uncertainty

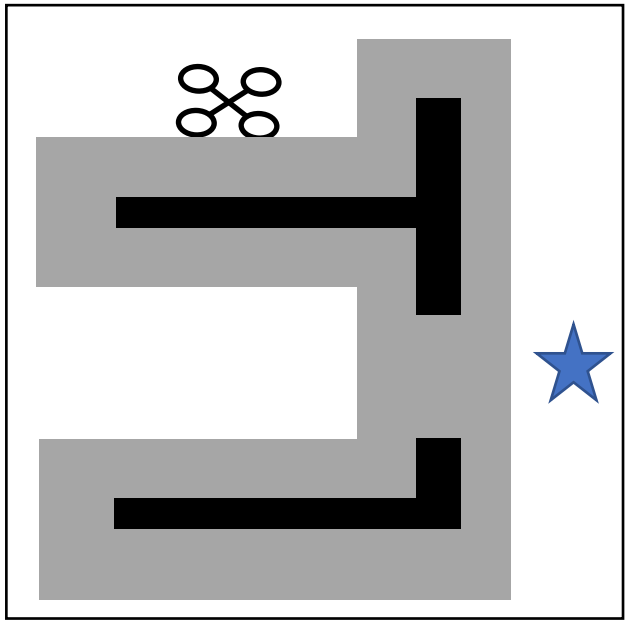
- Motivations:
  - Over-inflating obstacles is not a complete solution



Proper inflation



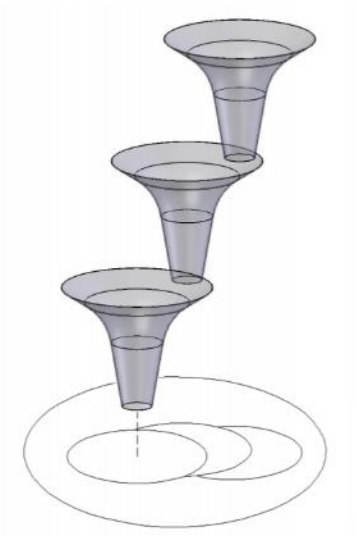
Okay over inflation



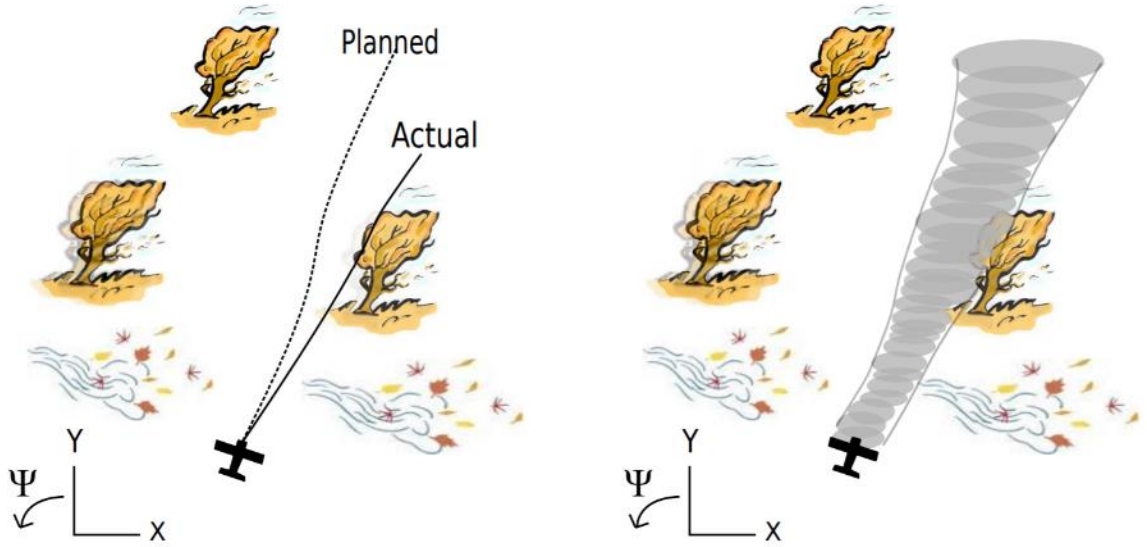
Bad over inflation

Extensions I: motion uncertainty

- Related work:
  - Reachability analysis: R. Tedrake and A.Majumdar 2017



R. Tedrake, MIT, 2009

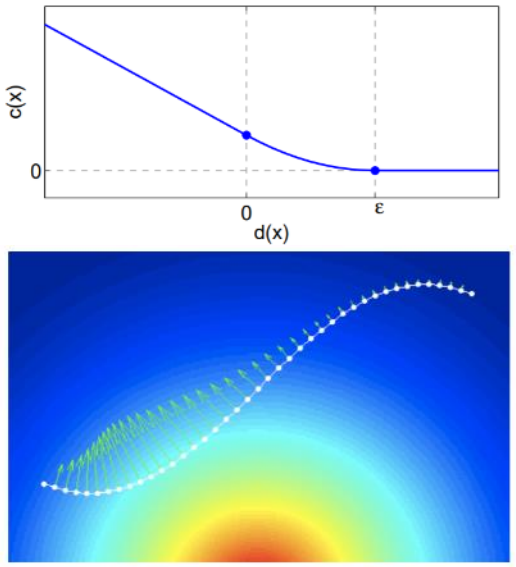


A. Majumdar and R. Tedrake, MIT, 2017

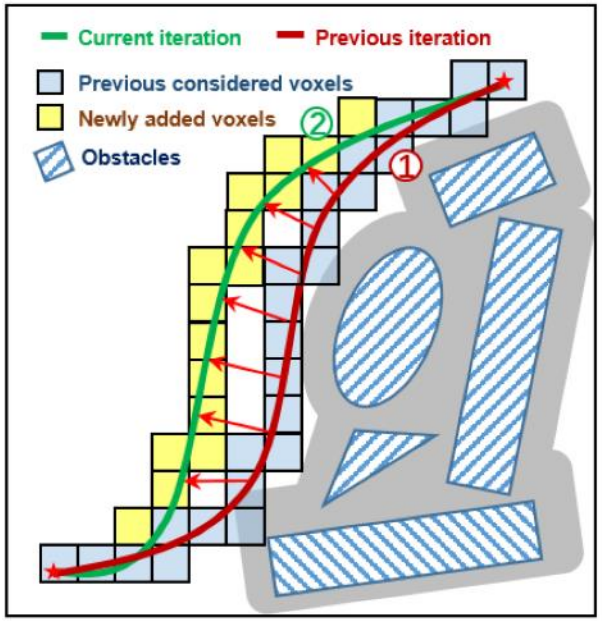
Drawbacks:
1. Computationally expensive
2. Too conservative

Extensions I: motion uncertainty

- Related work:
  - Gradient-based trajectory optimization: CHOMP 2009



CHOMP 2009



F. Gao and S. Shen, HKUST 2017

Drawbacks:
1. Rely on initial guess and time allocation
2. Undesired local minima

Extensions I: motion uncertainty

- Solution: Search with collision cost and tunnel constraints

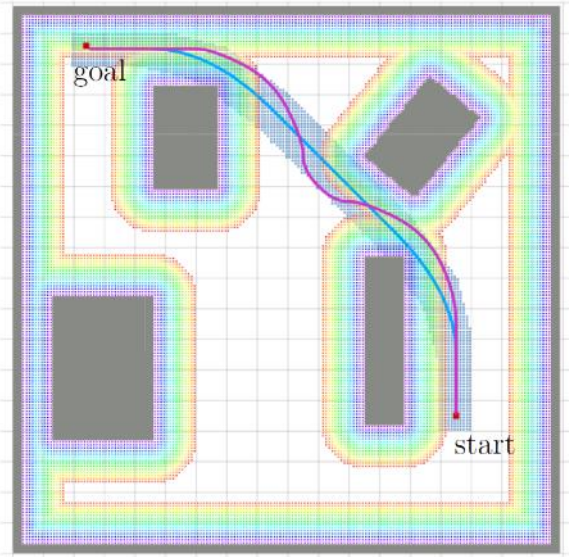
$$\arg\min_{\Phi} \; J_q + \rho_T T + \rho_c J_c \longrightarrow J_c(\Phi) = \int_{\Phi} U(s)\, ds.$$

$$s.t. \quad \dot{x}(t) = Ax(t) + Bu(t)$$

$$x(0) = x_0, \; x(T) \in \mathcal{X}^{goal}$$

$$x(t) \in \mathcal{T}(\Phi_0) \cap \mathcal{X}^{free}, \quad u(t) \in \mathcal{U}$$

$$\mathcal{T}(\Phi_0, r) = \Phi_0 \oplus D(r).$$



Nominal

Perturbed

Extensions I: motion uncertainty

- Solution: Search with collision cost and tunnel constraints

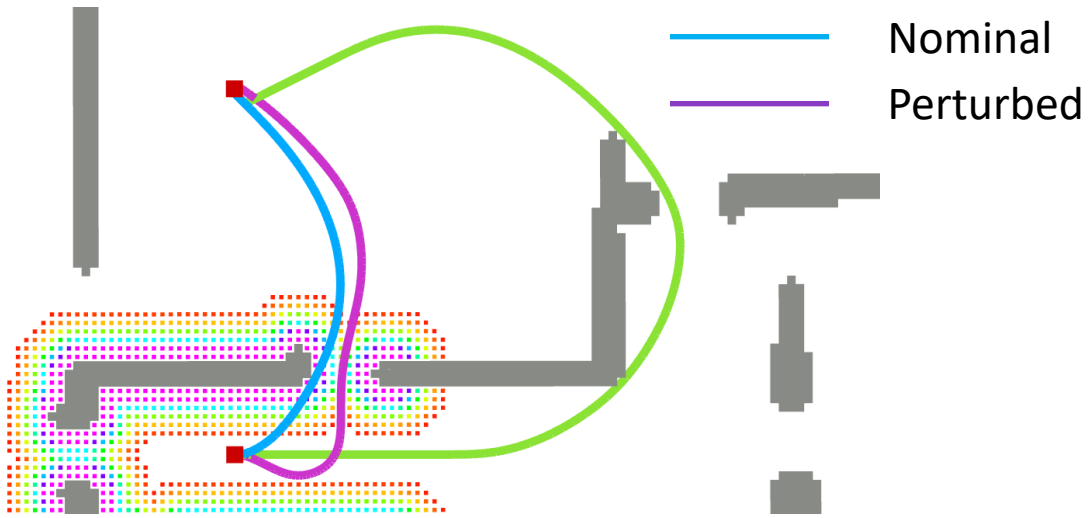
$$\arg\min_{\Phi} \; J_q + \rho_T T + \rho_c J_c \longrightarrow J_c(\Phi) = \int_{\Phi} U(s)\, ds.$$

$$s.t. \quad \dot{x}(t) = Ax(t) + Bu(t)$$

$$x(0) = x_0, \; x(T) \in \mathcal{X}^{goal}$$

$$x(t) \in \mathcal{T}(\Phi_0) \cap \mathcal{X}^{free}, \quad u(t) \in \mathcal{U}$$

$$\mathcal{T}(\Phi_0, r) = \Phi_0 \oplus D(r).$$



Extensions I: motion uncertainty

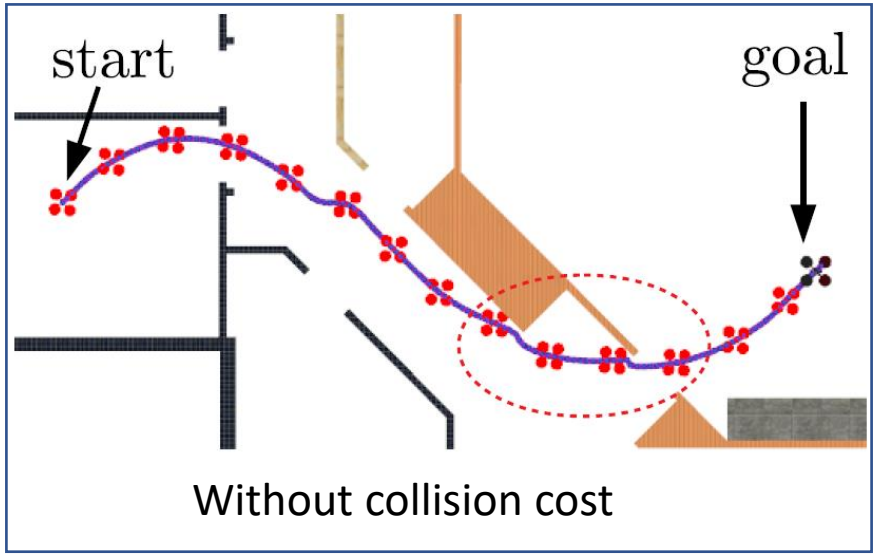
- Results:



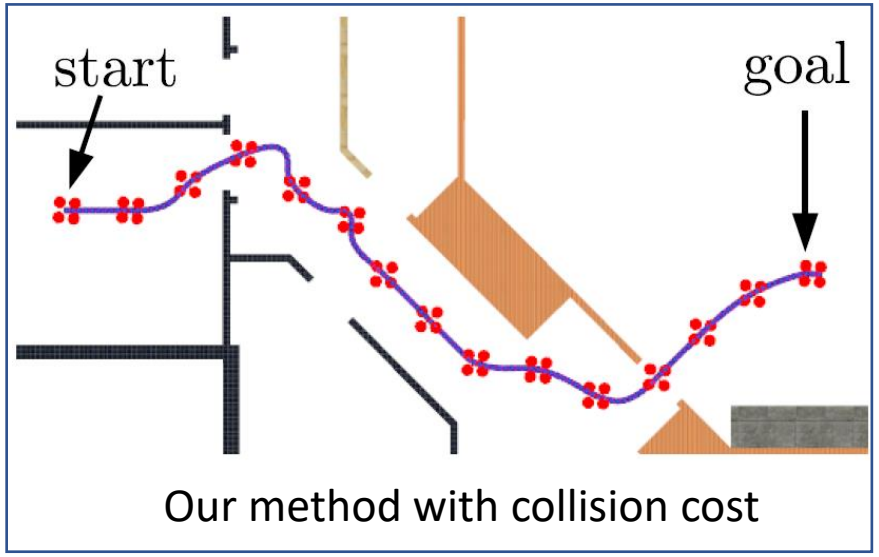
Going through the window



Going through the door

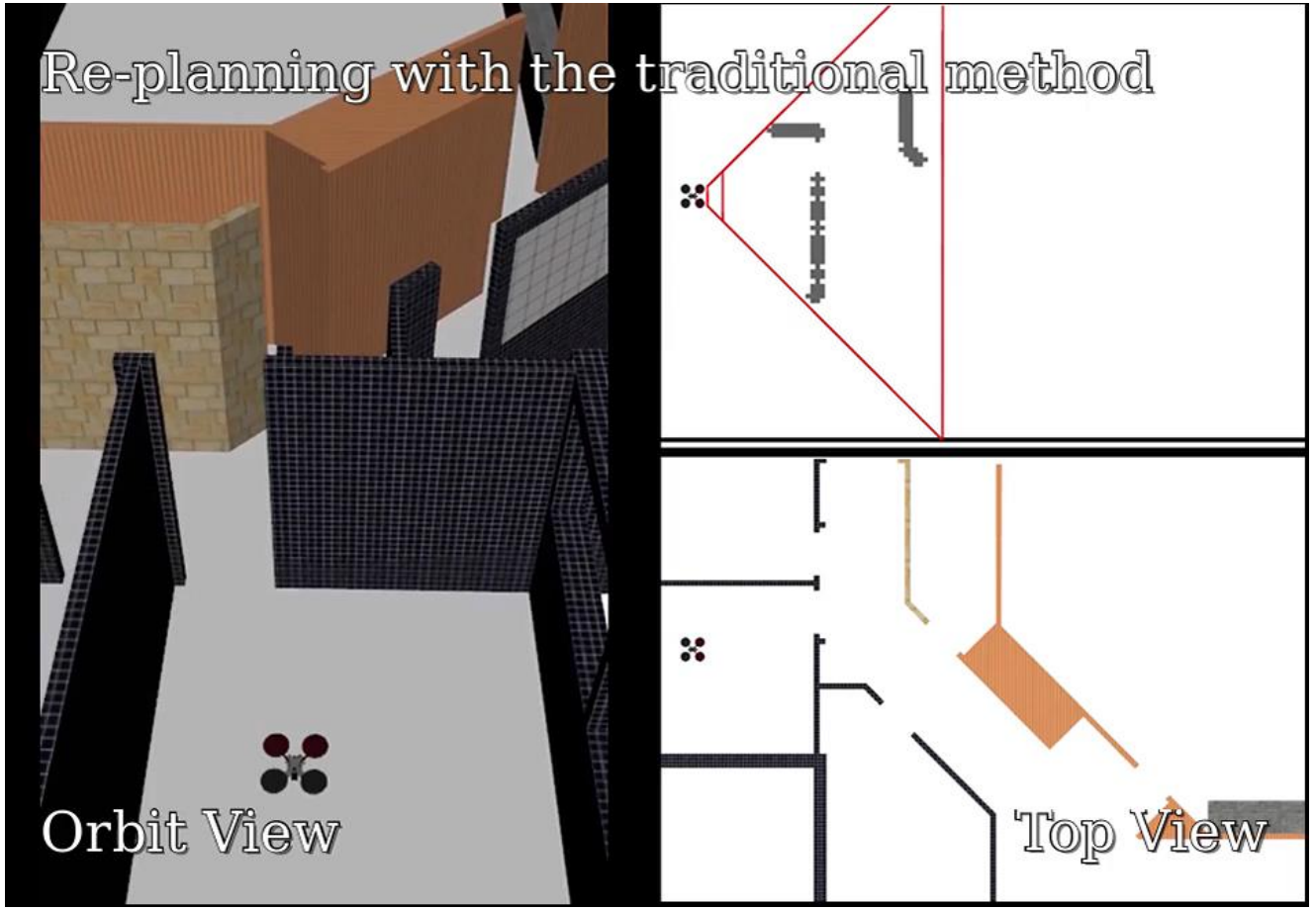


Without collision cost



Our method with collision cost

Extensions I: motion uncertainty



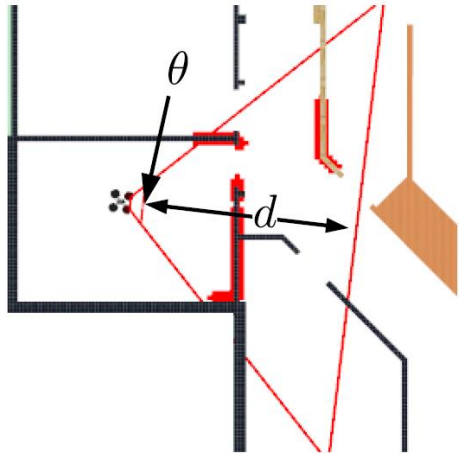
Extensions II: limited FOV

- Challenges in real world navigation:
  - Map is unknown
  - Sensor has limited FOV
- Desired yaw is non-linear and coupled:

$$\xi = \arctan v_y / v_x.$$

- Requirement for safe navigation:
  - the MAV should always move in the direction that is visible

$$|\psi(t) - \xi(t)| \leq \frac{\theta}{2},$$



Sensor model

Extensions II: limited FOV

- Our solution:
  - Add yaw cost and enforce a hard yaw constraint

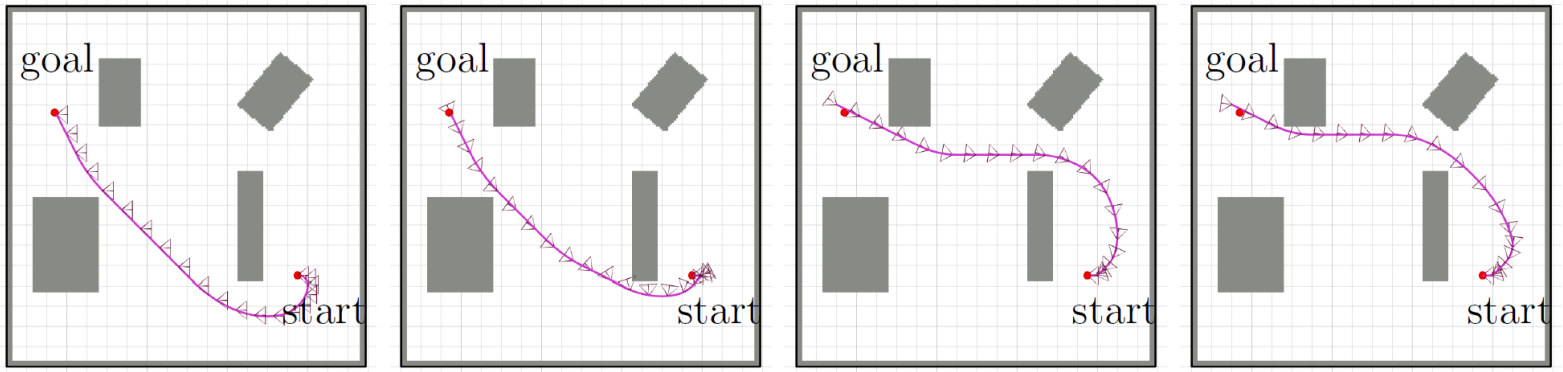
$$\arg\min_{\Phi} \; J_q + \rho_T T + \rho_\psi J_\psi \quad \rightarrow \quad J_\psi(\Phi) = \int_0^T [\psi(t) - \xi(t)]^2 dt,$$

$$s.t. \quad \dot{x}(t) = Ax(t) + Bu(t)$$

$$x(0) = x_0, \; x(T) \in \mathcal{X}^{goal}$$

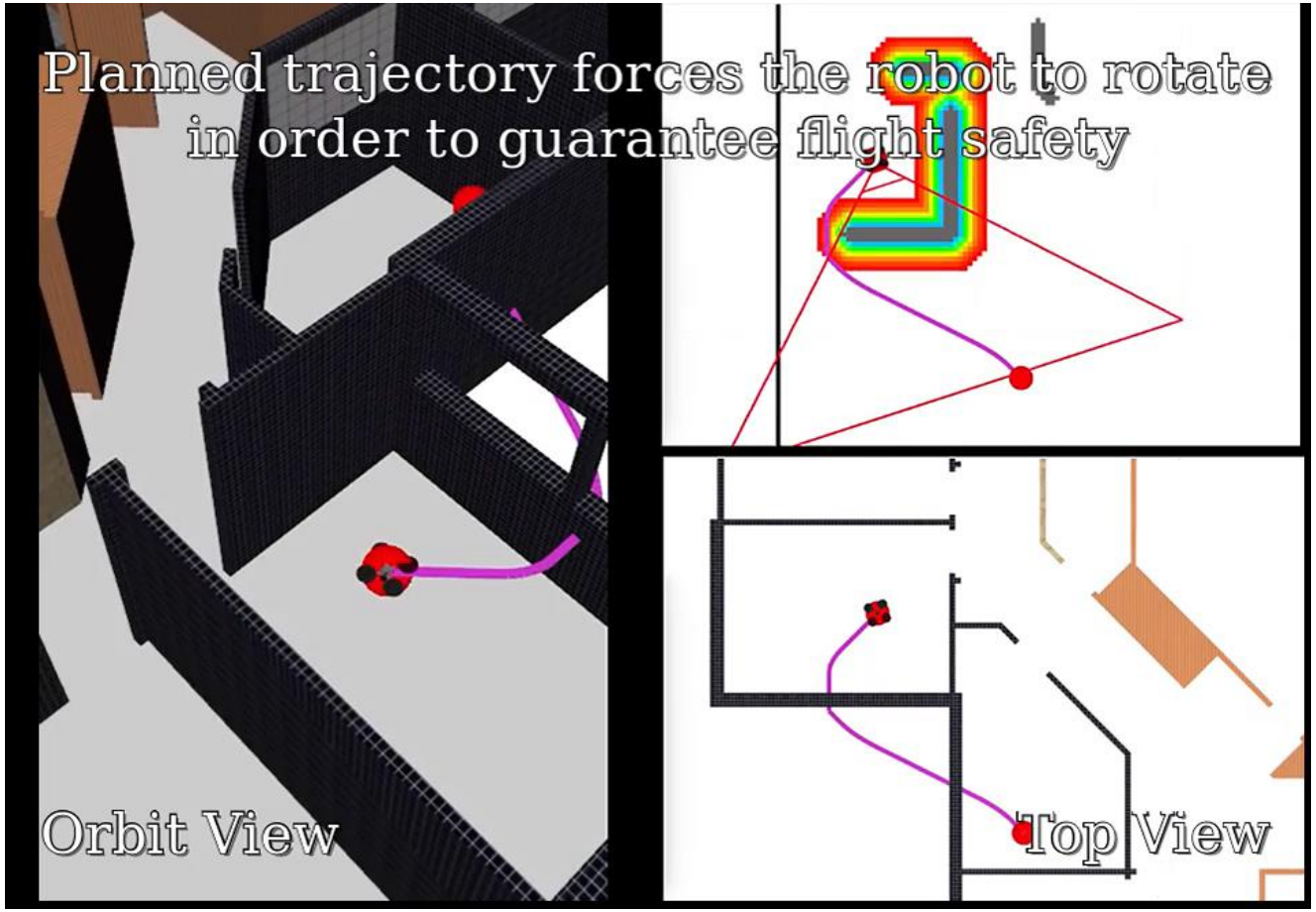
$$x(t) \in \mathcal{X}^{free}, \quad u(t) \in \mathcal{U}$$

$$\|\psi(t) - \xi(t)\| \leq \frac{\theta}{2}$$



(a) $\rho_\psi = 0, \; \theta = 2\pi.$ (b) $\rho_\psi = 1, \; \theta = 2\pi.$ (c) $\rho_\psi = 0, \; \theta = \pi/2.$ (d) $\rho_\psi = 1, \; \theta = \pi/2.$

Extensions II: limited FOV

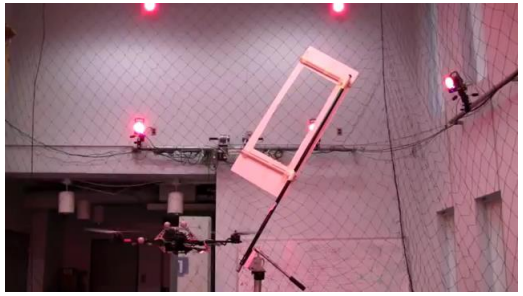


Extension III: SE(3)

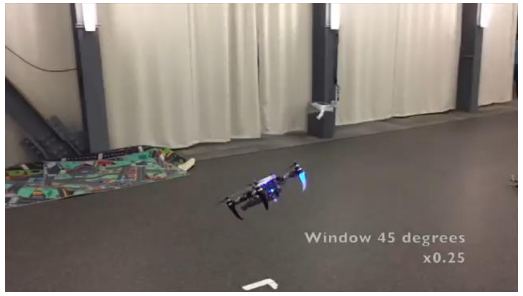
- Motivations:
  - Existing planning methods model quadrotor as a sphere or prism such that the attitude constraints are ignored



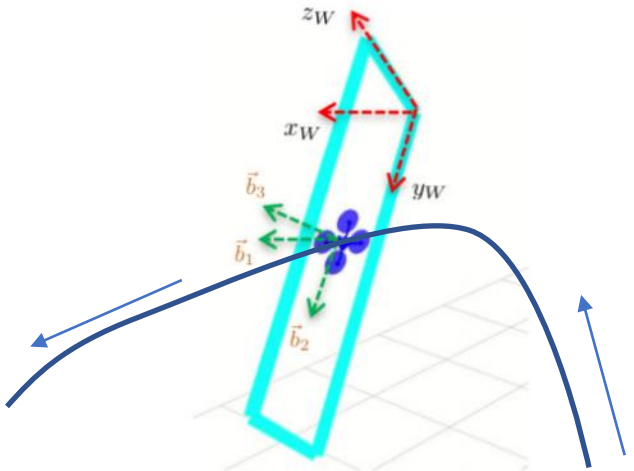
  - Quadrotor is able to agilely fly in 3D narrow spaces



Mellinger 2011

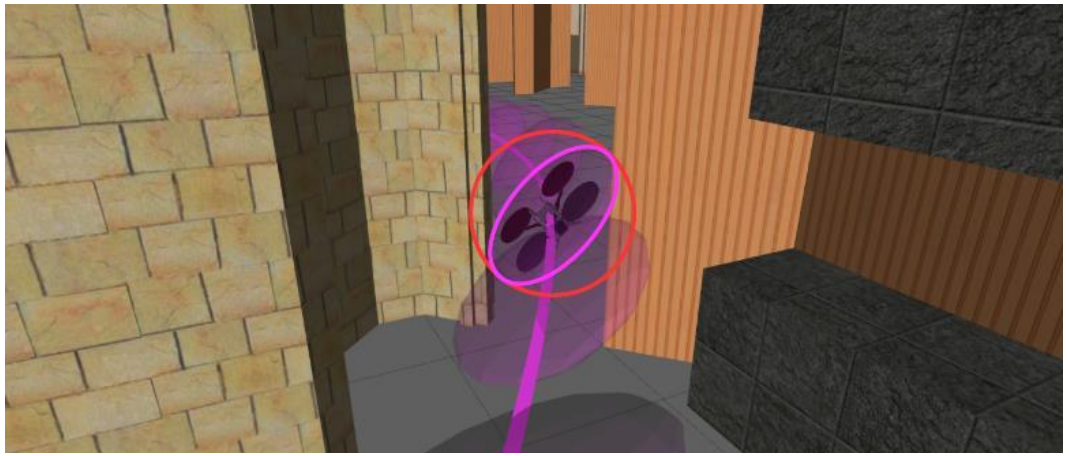


Loianno 2017



Extension III: SE(3)

- Our interests:
  - Planner that considers the robot's actual shape and dynamics
  - Plan in unstructured and cluttered environments such as collapsed building and debris



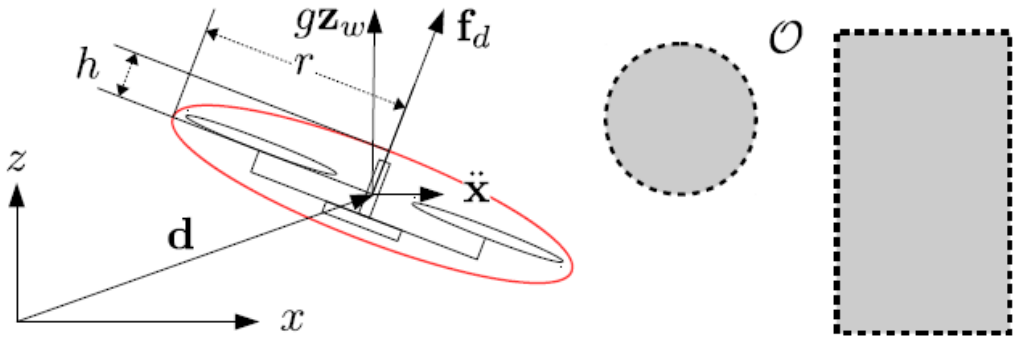
Liu 2018



FLA Phase 2, 2018

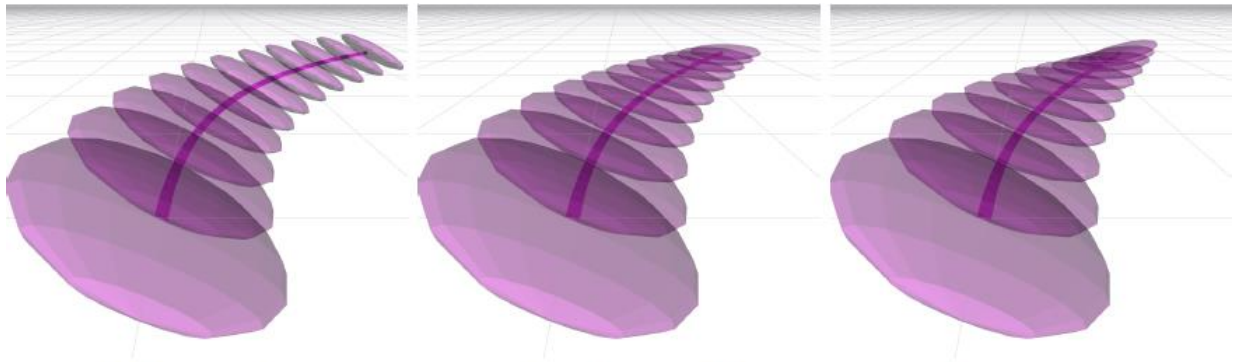
Extension III: SE(3)

- System dynamics in planning:



$$\mathbf{f}_d = \mathbf{a}_d + g\mathbf{z}_w.$$

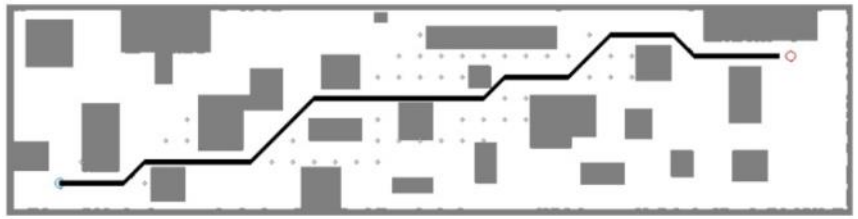
$$\mathbf{r}_3 = \mathbf{f}_d/\|\mathbf{f}_d\|, \quad \mathbf{r}_1 = \frac{\mathbf{r}_{2c} \times \mathbf{r}_3}{\|\mathbf{r}_{2c} \times \mathbf{r}_3\|}, \quad \mathbf{r}_2 = \mathbf{r}_3 \times \mathbf{r}_1$$



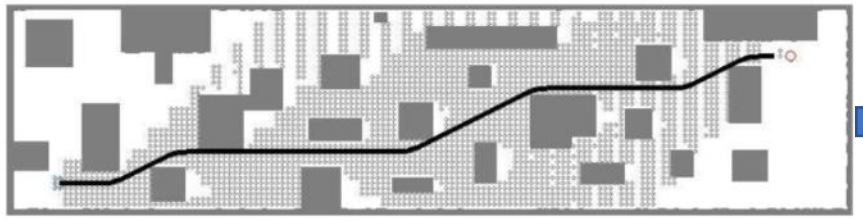
(a) Min-acc primitive. (b) Min-jrk primitive. (c) Min-snp primitive.

Extension III: SE(3)

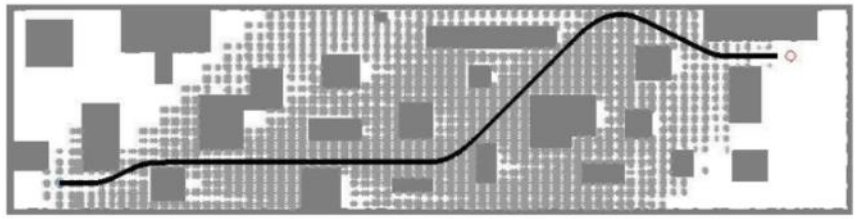
- Our solution:
  - Hierarchical search using trajectories in lower dimensional space as heuristics



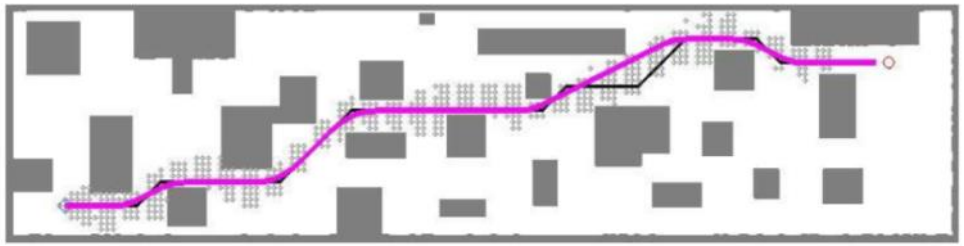
(a) $\Phi^1 : T^1 = 32s, J^1 = 42, t^1 = 2ms$



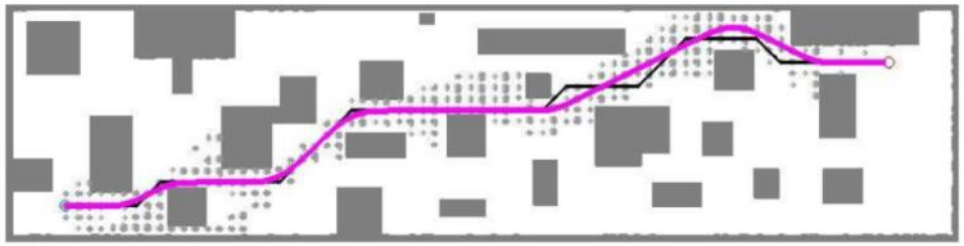
(b) $\Phi^2 : T^2 = 33s, J^2 = 2.25, t^2 = 60ms$



(c) $\Phi^3 : T^3 = 34s, J^3 = 3.75, t^3 = 1646ms$



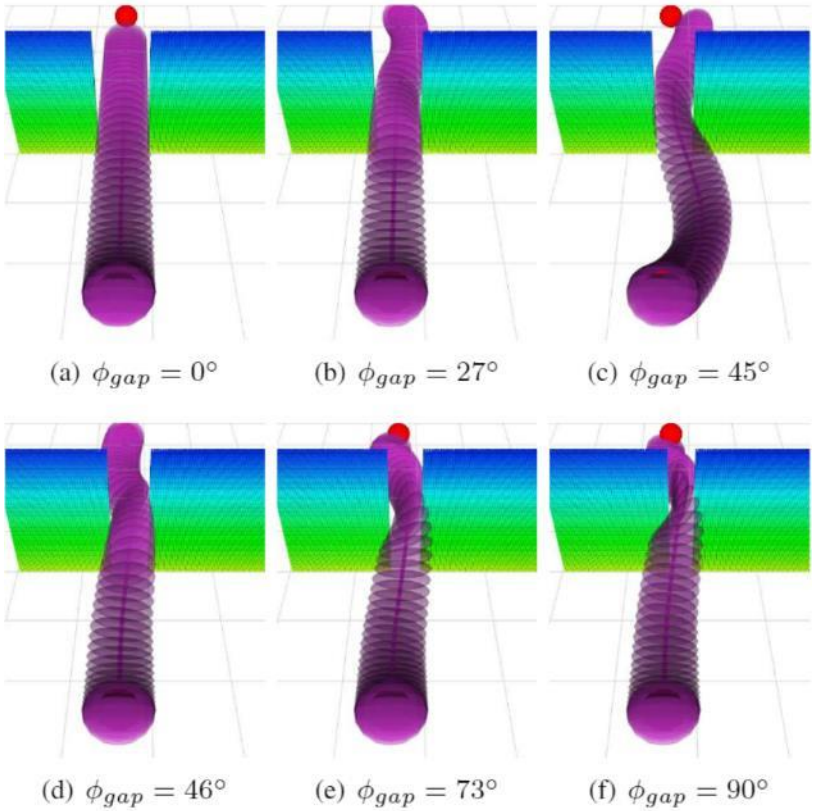
(a) $\Phi^2 : T^2 = 35s, J^2 = 3.0, t^2 = 11ms$



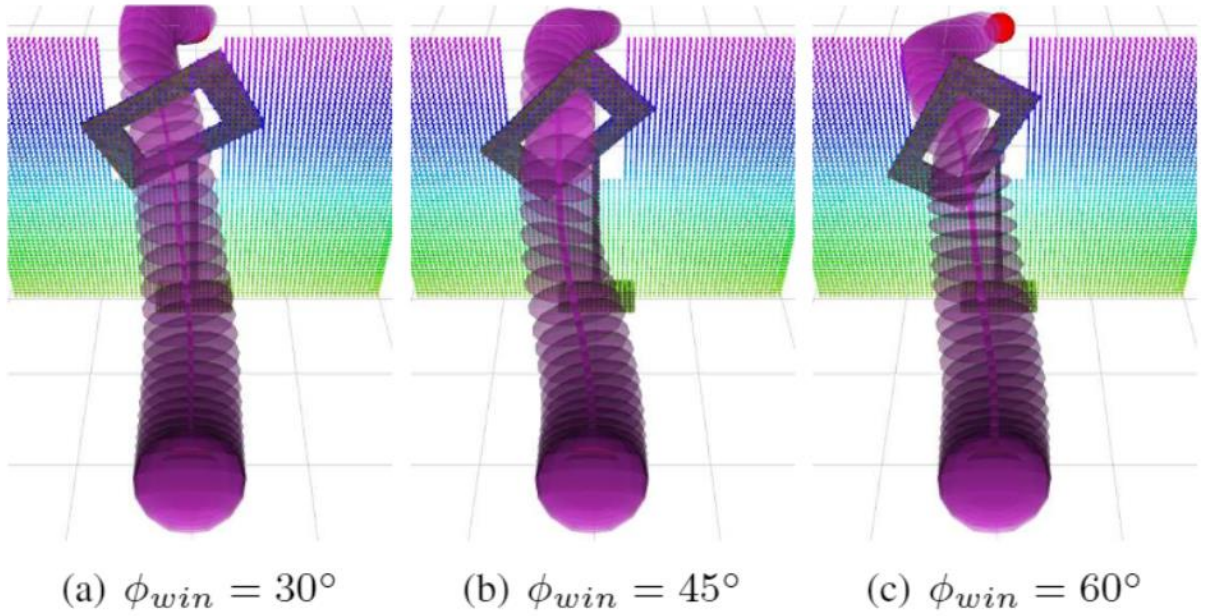
(b) $\Phi^3 : T^3 = 36s, J^3 = 4.25, t^3 = 98ms$

Extension III: SE(3)

- Example I:



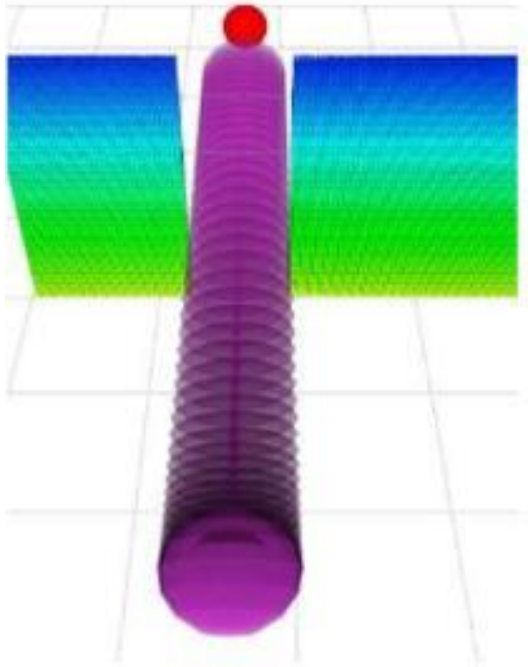
Shrinking doors



Windows

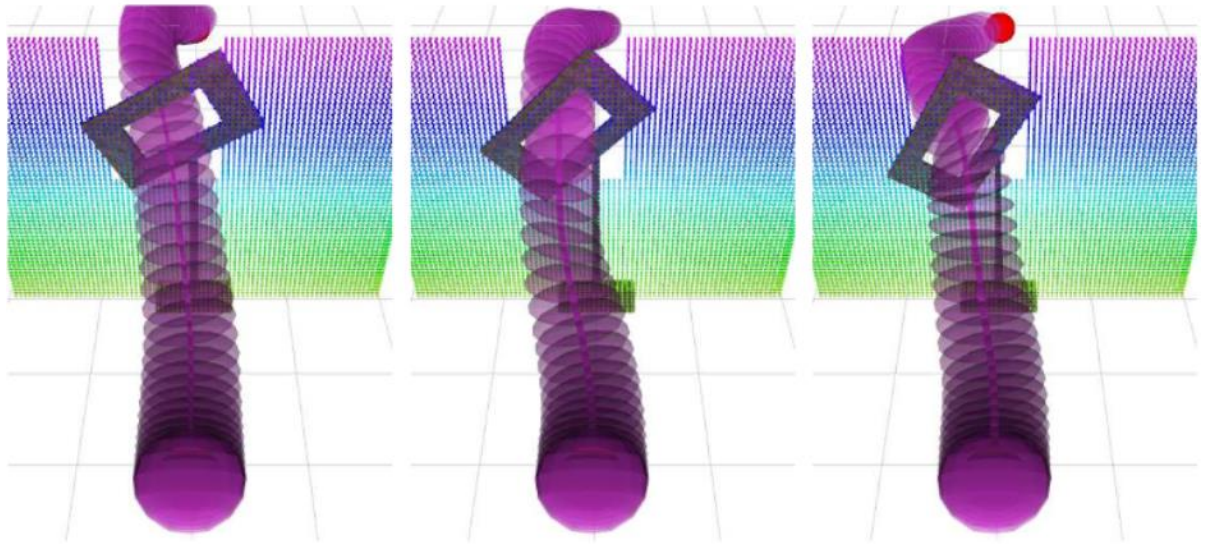
Extension III: SE(3)

- Example I:



(a) $\phi_{gap} = 0°$

Shrinking doors

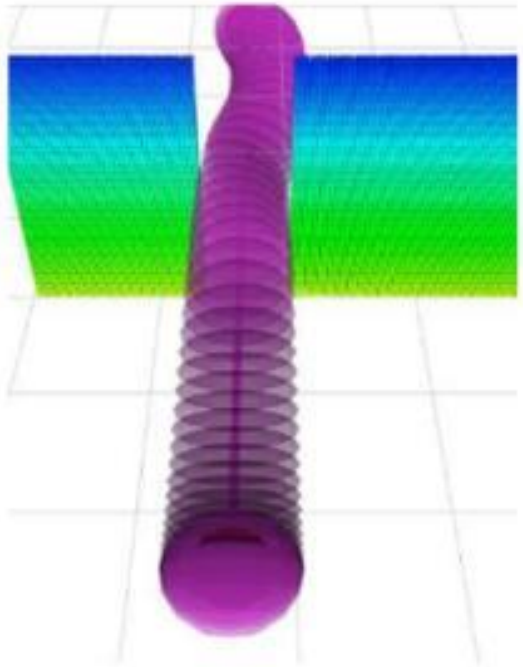


(a) $\phi_{win} = 30°$ (b) $\phi_{win} = 45°$ (c) $\phi_{win} = 60°$

Windows

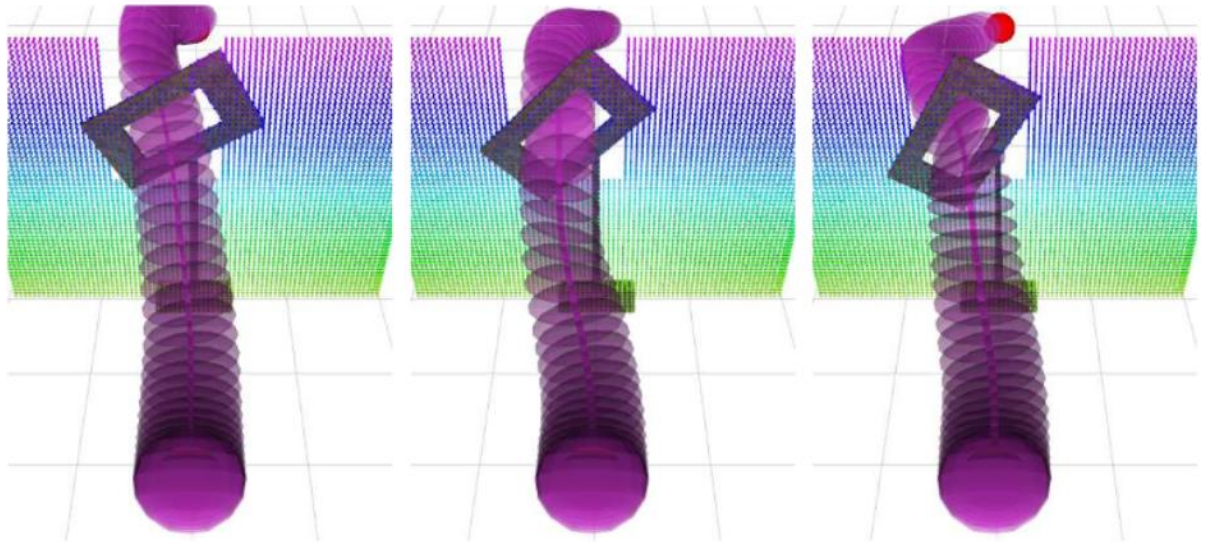
Extension III: SE(3)

- Example I:



(b) $\phi_{gap} = 27°$

Shrinking doors

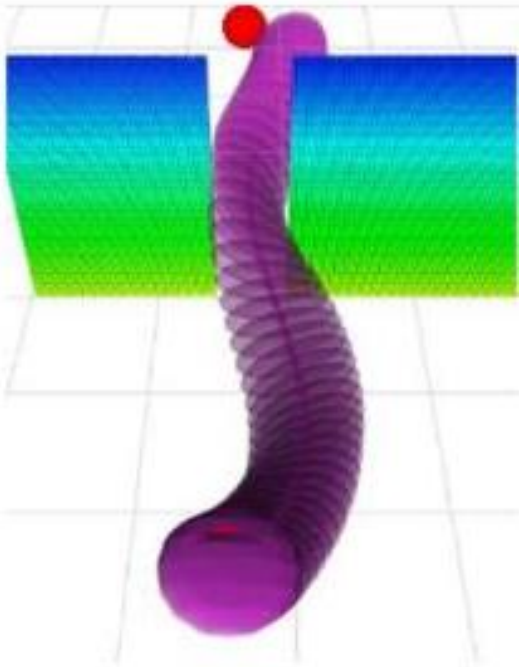


(a) $\phi_{win} = 30°$ (b) $\phi_{win} = 45°$ (c) $\phi_{win} = 60°$

Windows

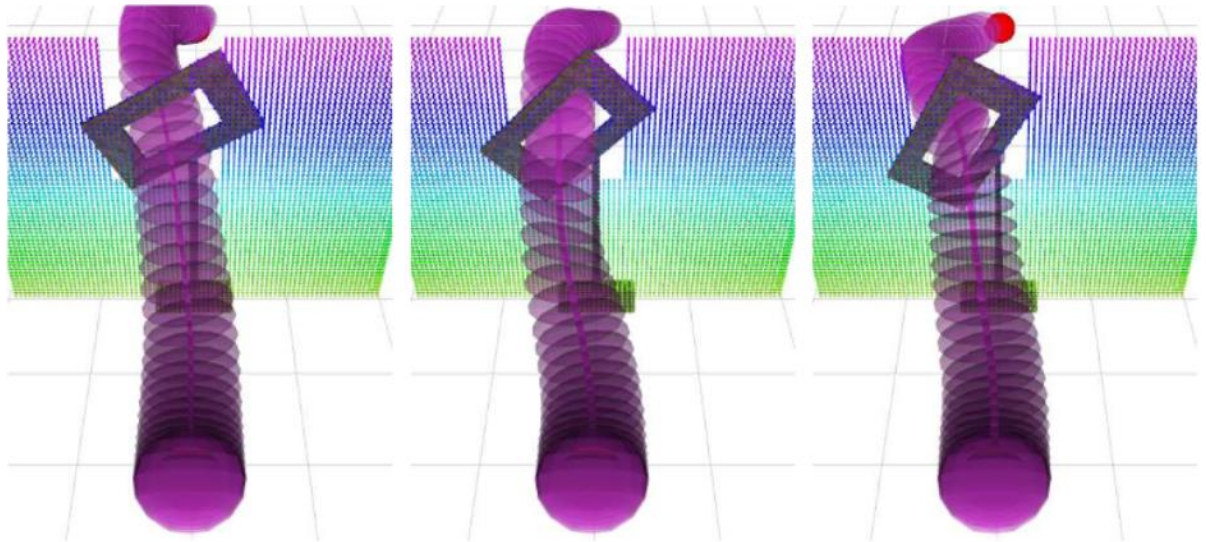
Extension III: SE(3)

- Example I:



(c) $\phi_{gap} = 45°$

Shrinking doors

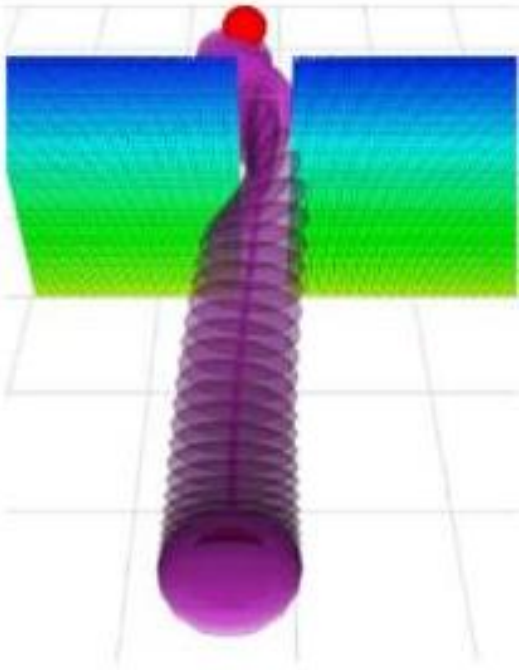


(a) $\phi_{win} = 30°$ (b) $\phi_{win} = 45°$ (c) $\phi_{win} = 60°$

Windows

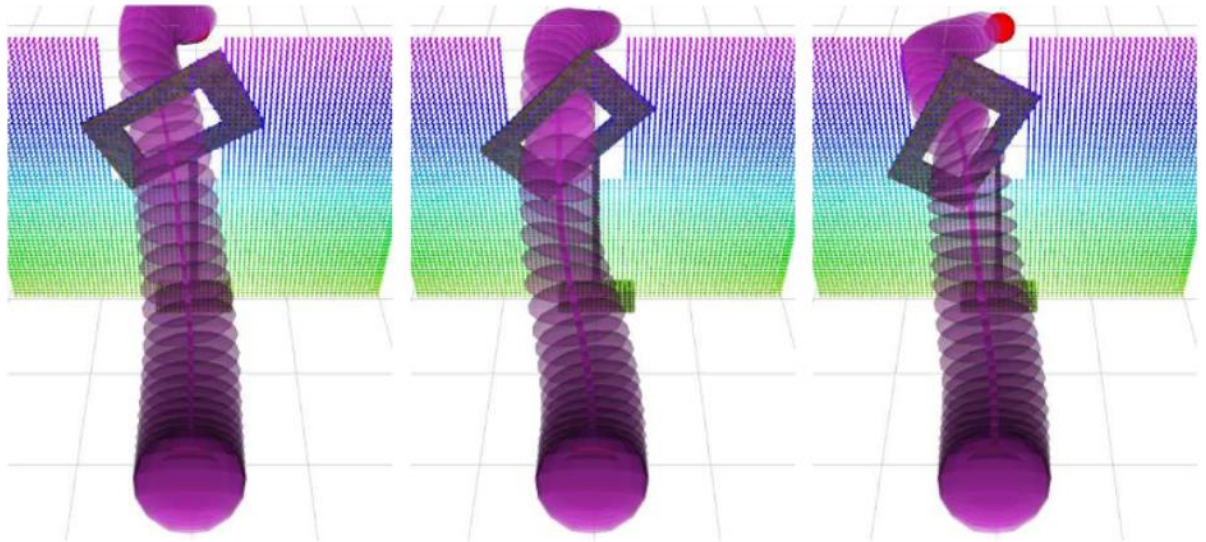
Extension III: SE(3)

- Example I:



(f) $\phi_{gap} = 90°$

Shrinking doors

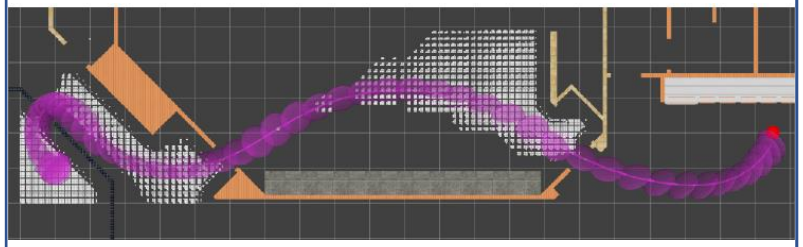


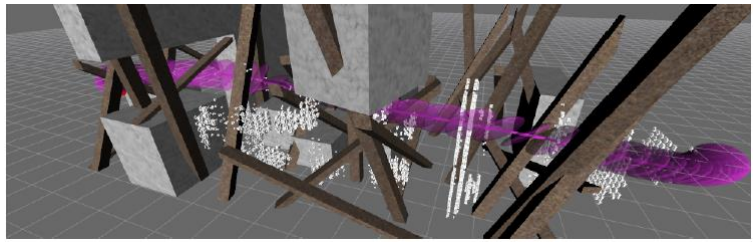
(a) $\phi_{win} = 30°$ (b) $\phi_{win} = 45°$ (c) $\phi_{win} = 60°$

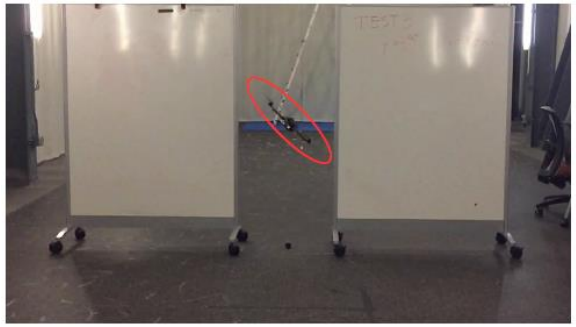
Windows

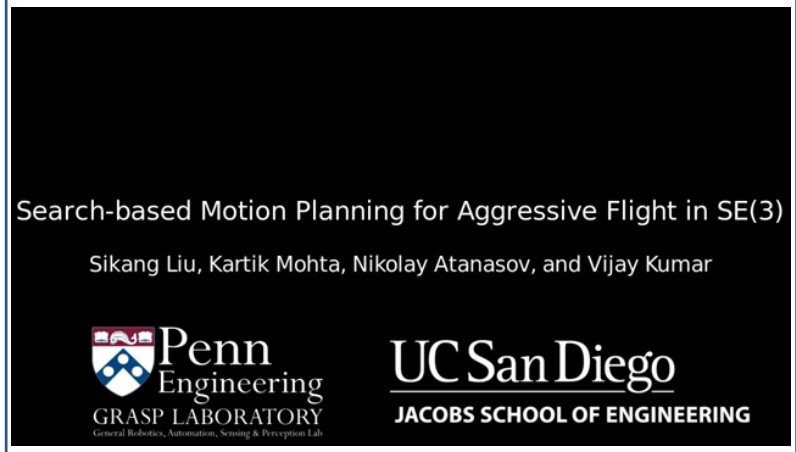
Extension III: SE(3)

- Example II:





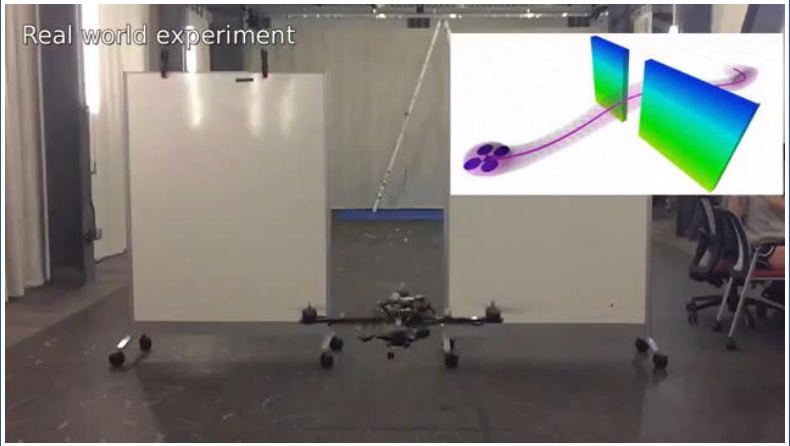




Office Environment



3D Unstructured Environment



Experiment

Extension IV: Dynamic Environments

- List of contents:
  - Trajectory planning with moving obstacles
  - Incremental trajectory planning
  - Multi-robot planning
- General problem formulation:

$$\arg\min_{\Phi} \; J_q(\Phi) + \rho_T T$$

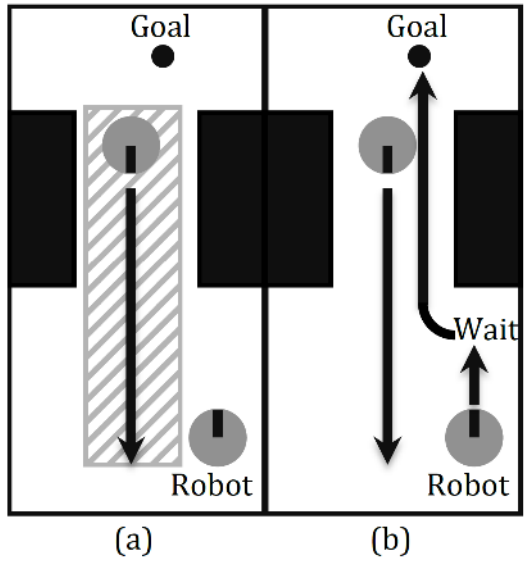
$$s.t. \;\; \dot{x}(t) = Ax(t) + Bu(t),$$

$$x(0) = x_0, \quad x(T) \in \mathcal{X}^{goal},$$

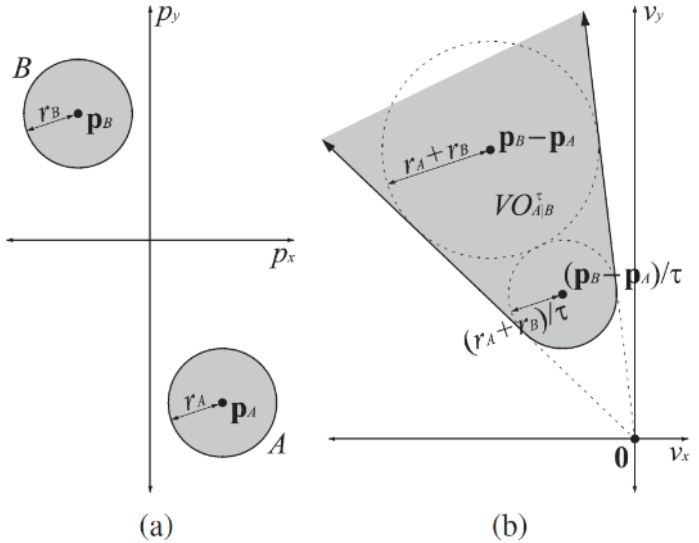
Collision constraints → $x(t) \in \mathcal{X}^{free}, \quad u(t) \in \mathcal{U}.$ $\quad x(t) \in \mathcal{X}_s^{free}, \text{ and } x(t) \in \mathcal{X}_d^{free}(t).$

Planning in Dynamic Environments I: moving obstacles

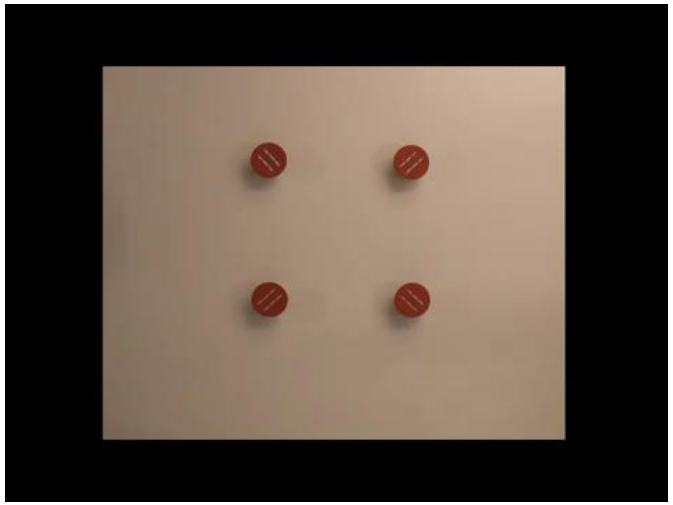
- Related work:
  - Re-plan based approaches: D*/D* Lite, incremental RRT* and etc
  - Safe interval
  - Velocity obstacle (VO) based reactive collision avoidance



M. Likhachev, CMU, 2011



J. Berg, University of Utah, 2009



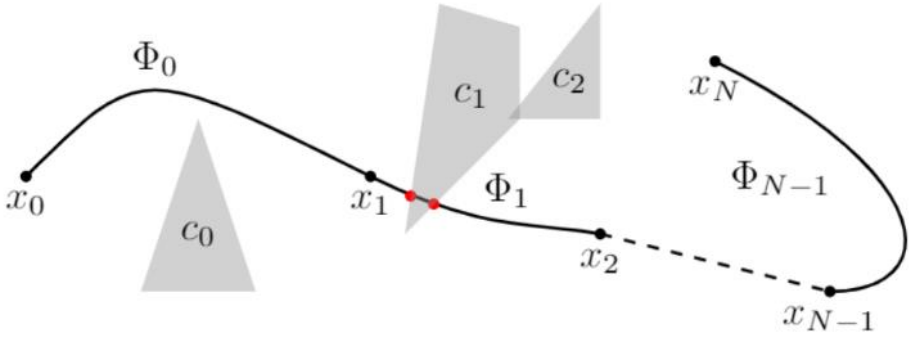
ORCA for 4 robots, ETH, ASLTeam

Planning in Dynamic Environments I: moving obstacles

- Collision checking:

Proposition 4. *A trajectory segment $\Phi_i(t)$ intersects a polyhedron c composed of half-spaces described by $\mathbf{A}$ and $\mathbf{b}$ if and only if*

$$\exists\ t_c \in [0, \Delta t_i] \quad s.t. \quad \mathbf{A}^\mathsf{T} \Phi_i(t_c) \leq \mathbf{b} \tag{6.1}$$

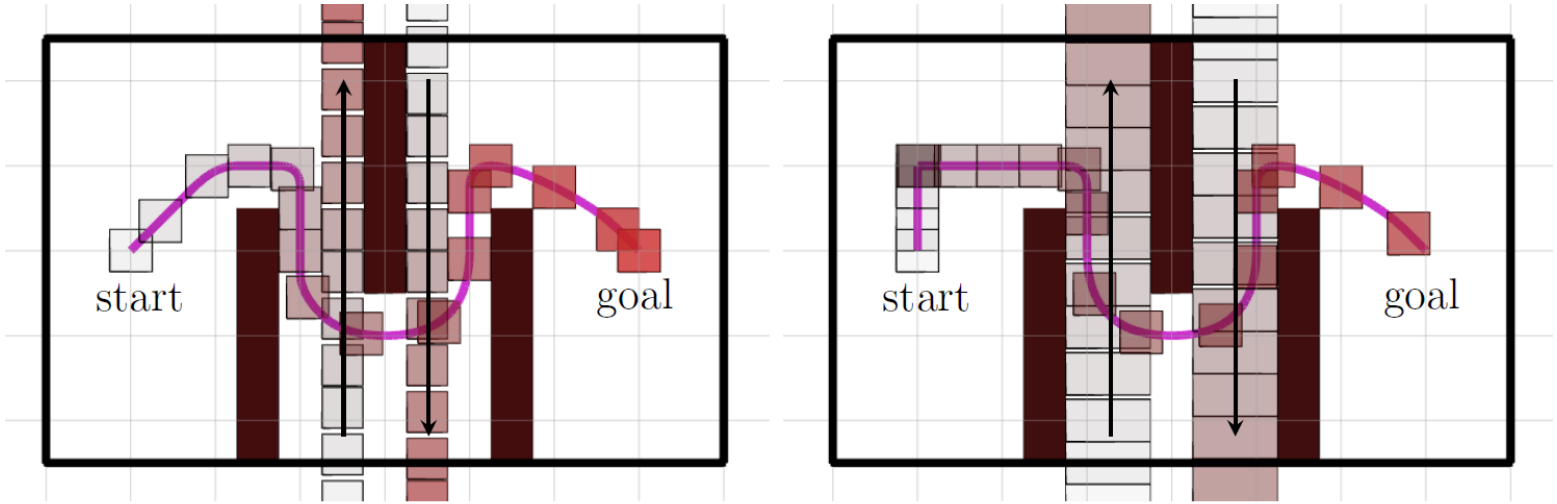


Planning in Dynamic Environments I: moving obstacles

- Linear Velocity Polyhedron (LVP):

$$c = \bigcap_{j=0}^{M-1} h_j = \{p \mid \mathbf{A}^\mathsf{T} p \leq \mathbf{b},\ p \in \mathbb{R}^m\},$$

$$a_j(t) = a_{j,0},\ b_j(t) = b_{j,0} + a_{j,0}^\mathsf{T} \cdot v_c t.$$



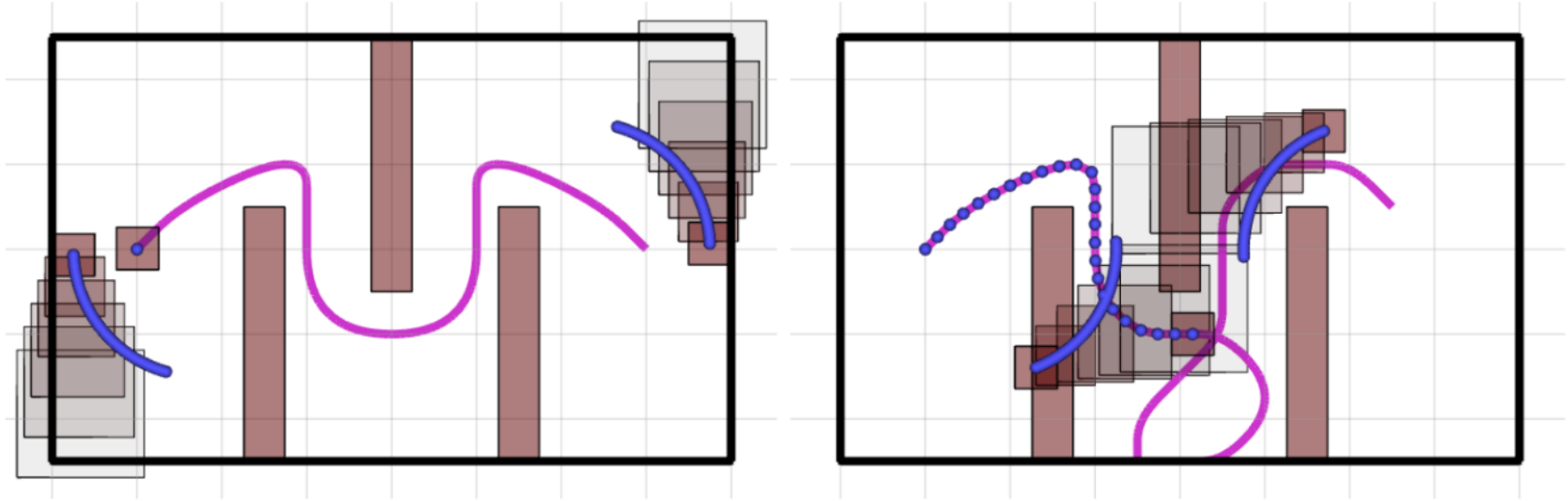
(a) Configuration 1. (b) Configuration 2.

Planning in Dynamic Environments I: moving obstacles

- Uncertainty in LVP:

$$b_j(t) = b_{j,0} + a_{j,0}^{\mathsf{T}} \cdot v_c t.$$

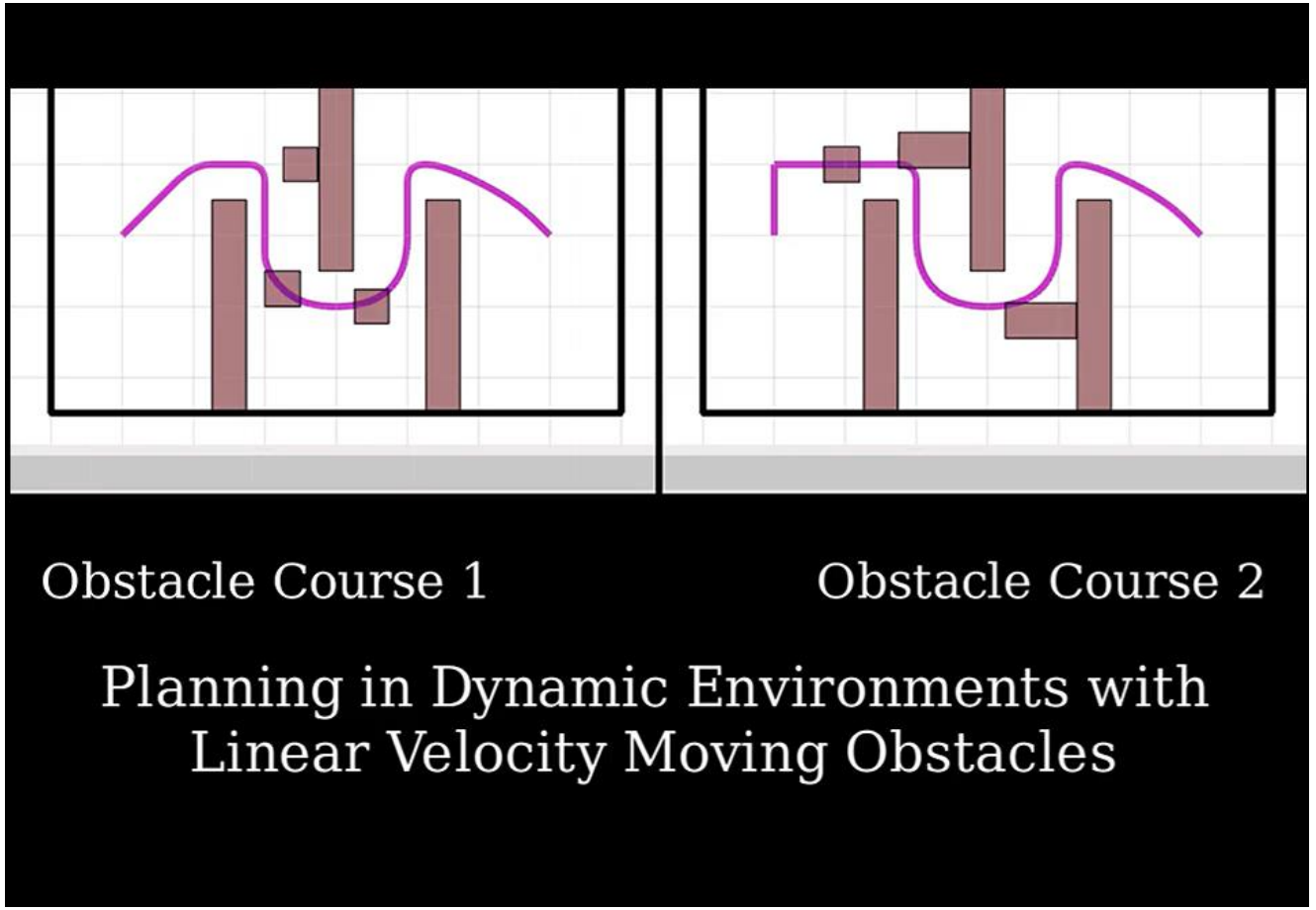
$$b_j(t) = b_{j,0} + (a_{j,0}^{\mathsf{T}} \cdot v_c + \|a_{j,0}\| v_e) t.$$



(a) Plan epoch 0.

(b) Plan epoch 12.

Planning in Dynamic Environments I: moving obstacles

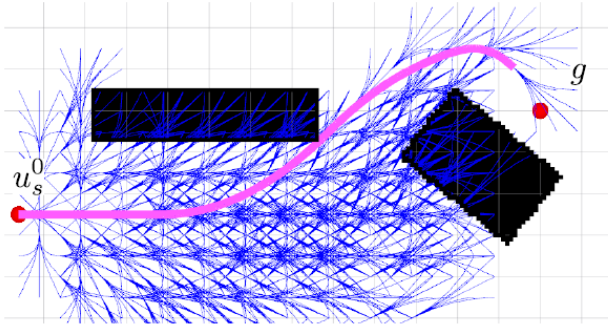


Planning in Dynamic Environments II: incremental re-plan

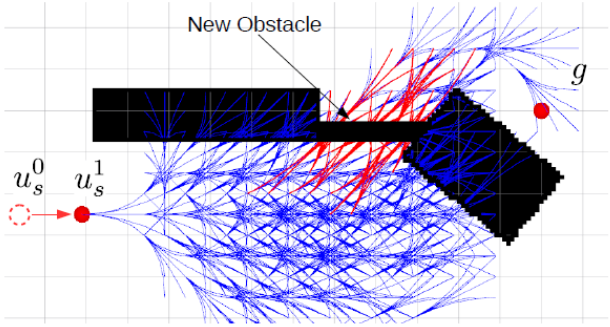
- Reuse information from previous search:
  - Lifelong Planning A* (LPA*)
  - Graph pruning

Algorithm 5 *Incremental Trajectory Planning.*

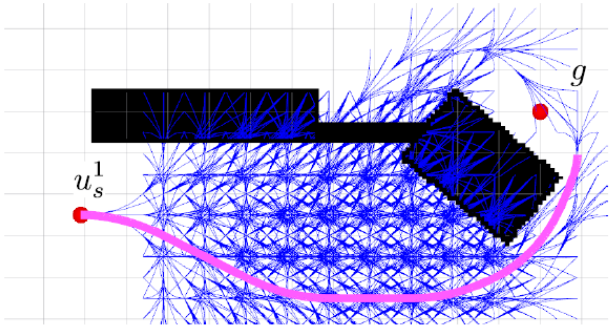
```
1: procedure Main()
2:     InitializeGraph();
3:     u_g ← g, InitializeState(u_g);
4:     while u_s ≉ g do
5:         u_g, Φ ← ComputeTrajectory(u_g);
6:         u_s ← Φ(τ);
7:         process Graph Pruning with the new u_s;
8:         while robot approaching u_s do
9:             for all e(u, v) that has changed cost do
10:                UpdateState(v);
```



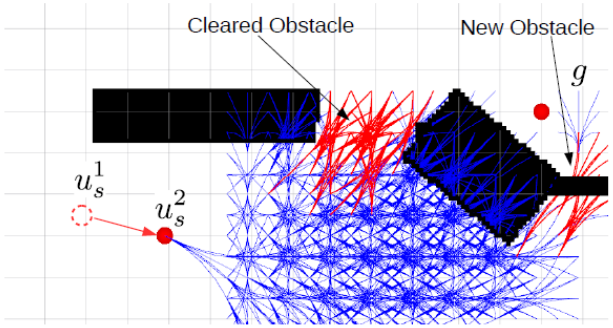
(a) Planning iteration 0



(b) Prune and update graph



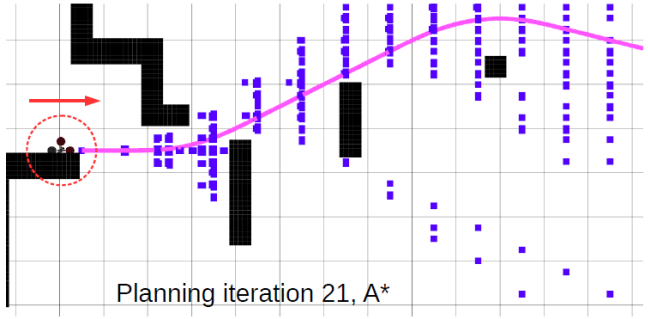
(c) Planning iteration 1

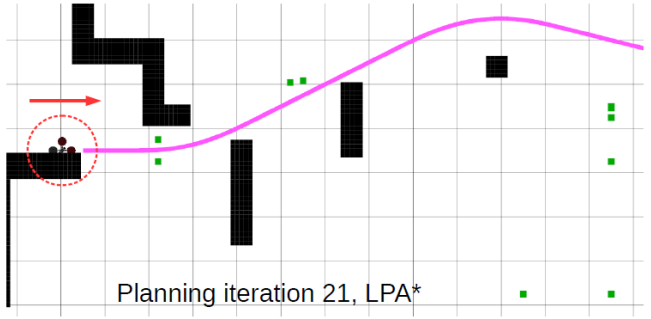


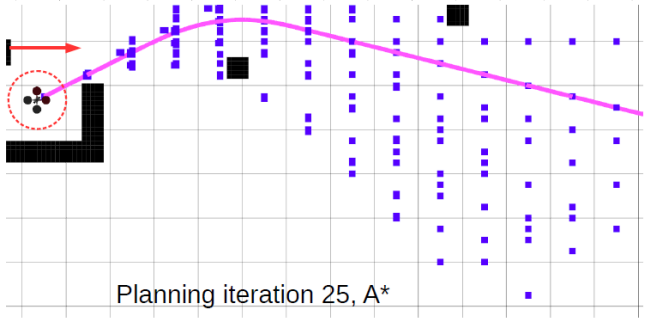
(d) Prune and update graph

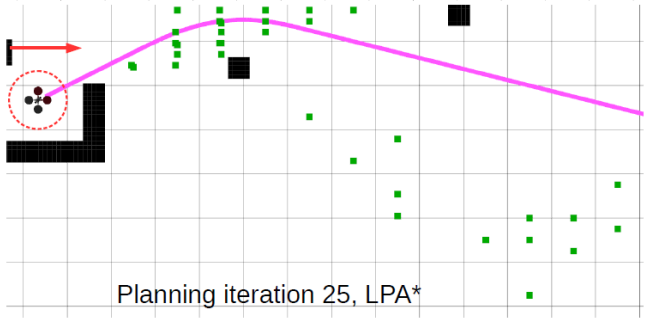
Planning in Dynamic Environments II: incremental re-plan

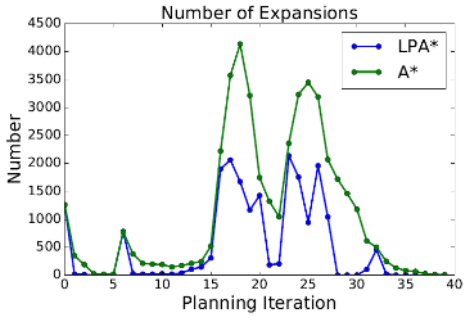
- Evaluation:

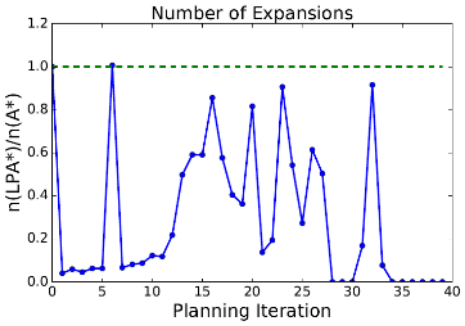


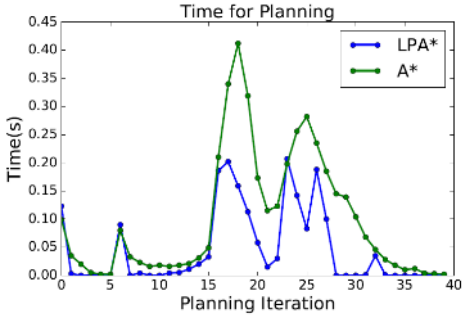


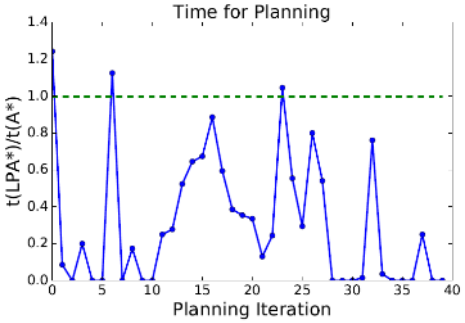






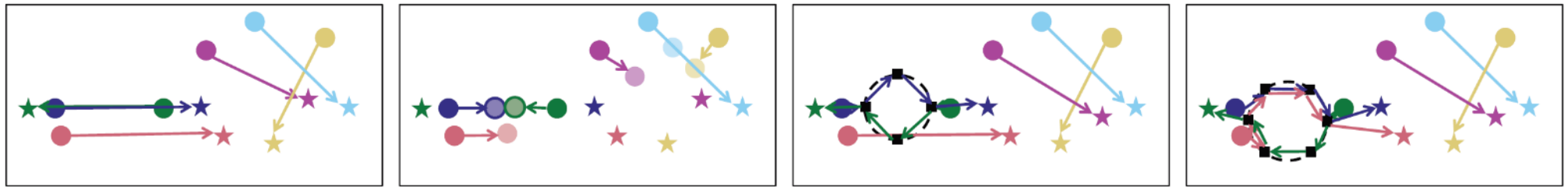




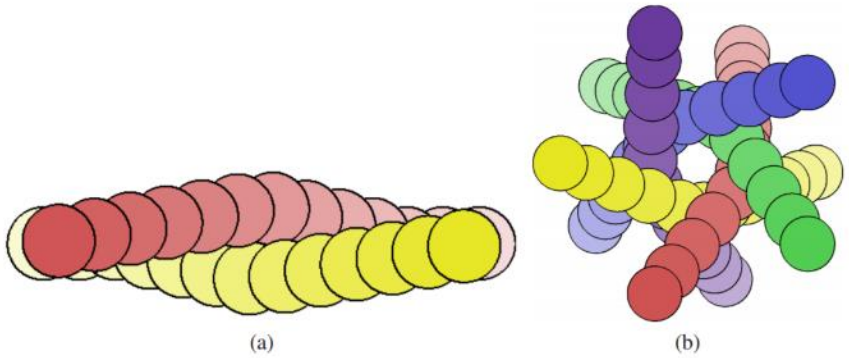


Planning in Dynamic Environments III: multi-robot planning

- Related work:
  - Goal switching: CAPT 2014
  - Decentralized methods: ORCA 2013



S. Tang and V. Kumar, Upenn, 2018



ORCA, 2010

Planning in Dynamic Environments III: multi-robot planning

- Our interests:
  - Plan trajectories for a team of robots with assigned goals
  - Fast, optimal and complete algorithm
  - Centralized and decentralized scenes
- Our solution:
  - Apply search-based trajectory planning
  - Sequential planning with priority
  - Decentralized without priority

Planning in Dynamic Environments III: multi-robot planning

- Inter-robot collision checking:
  - Model robot as non-linear moving polyhedron:

$$c_i(t) = \{p \mid \mathbf{A}_i^{\mathsf{T}}(p - {}^i\Phi(t)) \leq \mathbf{b}_i\}.$$

  - Collision checking between two robots:

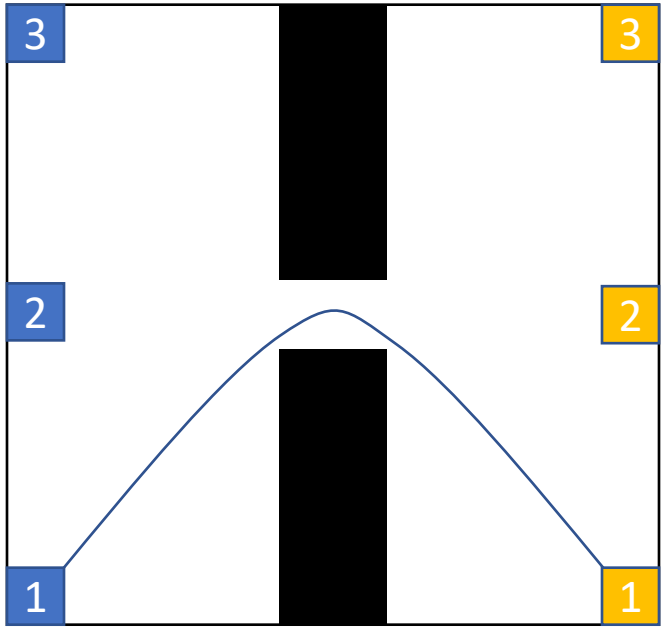
$${}^i\Phi(t) \cap [c_{i,0} \oplus c_j(t)] = \emptyset,$$

  - Collision checking against other robots:

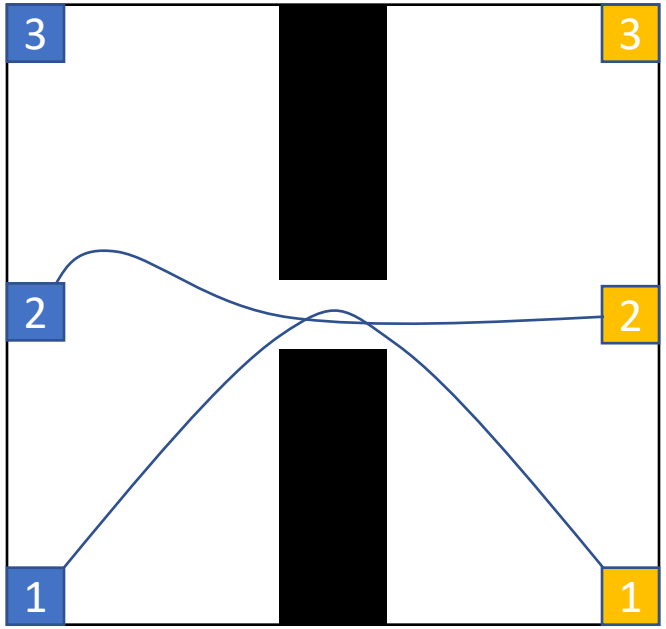
$${}^i\Phi(t) \cap \bigcup_{j=0}^{i-1} [c_{i,0} \oplus c_j(t)] = \emptyset.$$

Planning in Dynamic Environments III: multi-robot planning

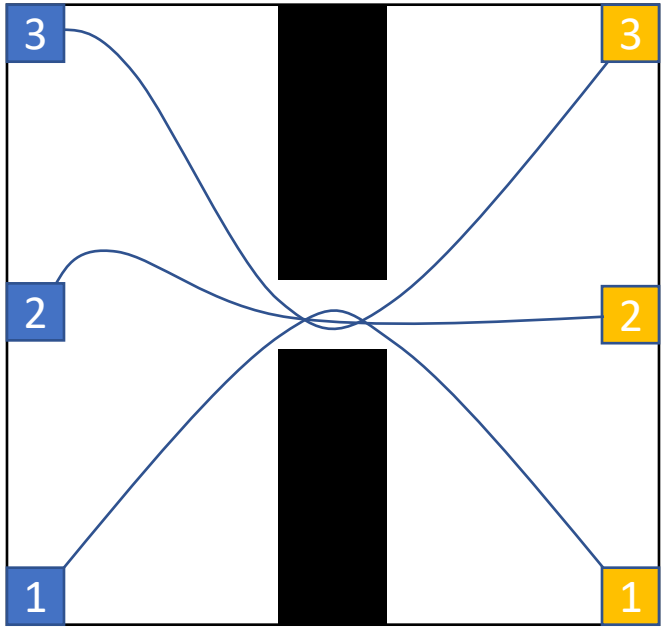
- Sequentially planning:



First plan



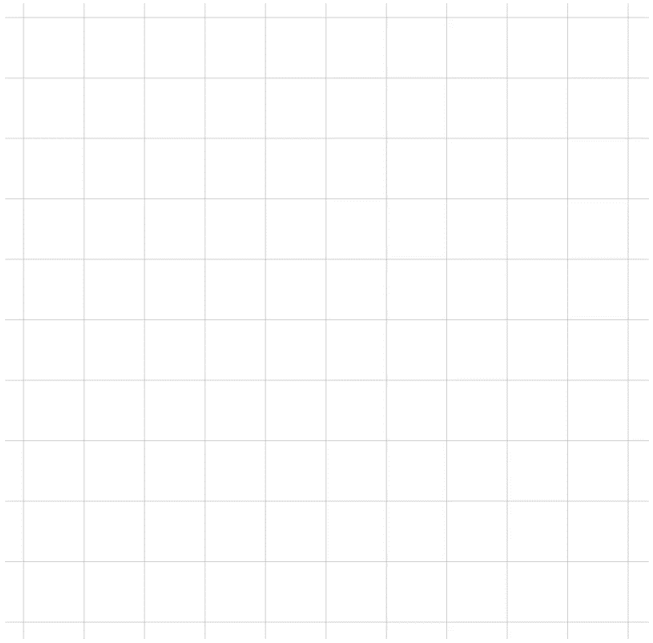
Second plan



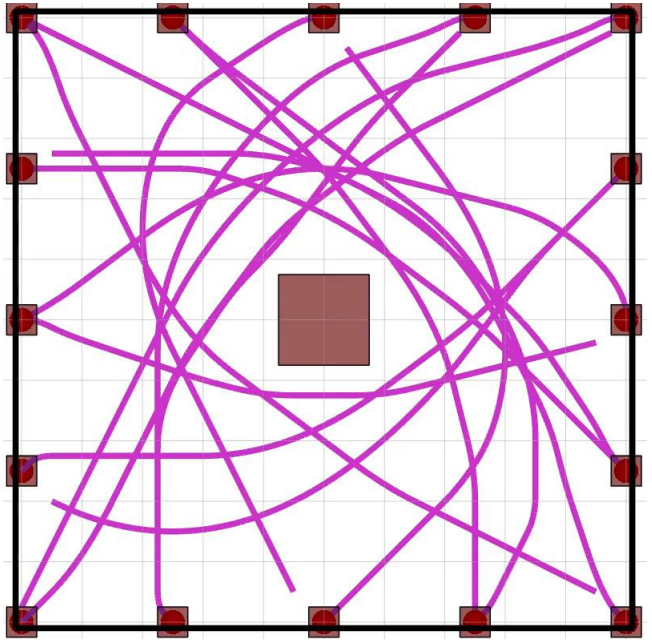
Third plan

Planning in Dynamic Environments III: multi-robot planning

- Sequentially planning:



Tunnel Configuration



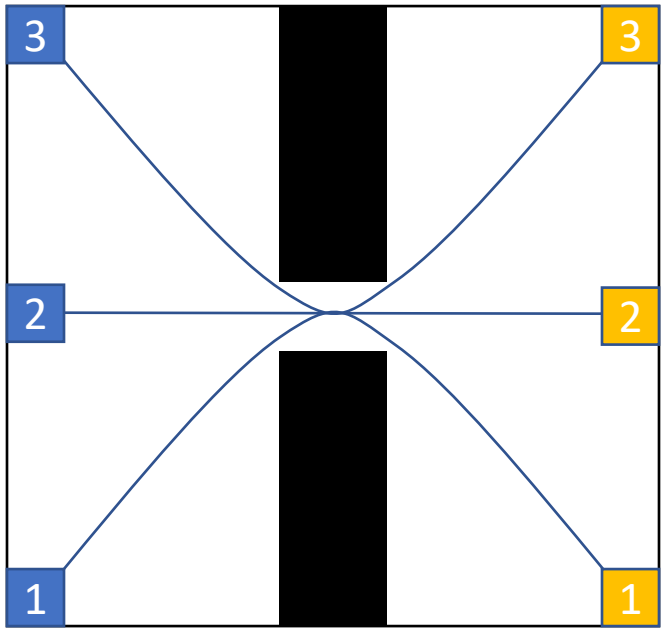
Star Configuration

Planning in Dynamic Environments III: multi-robot planning

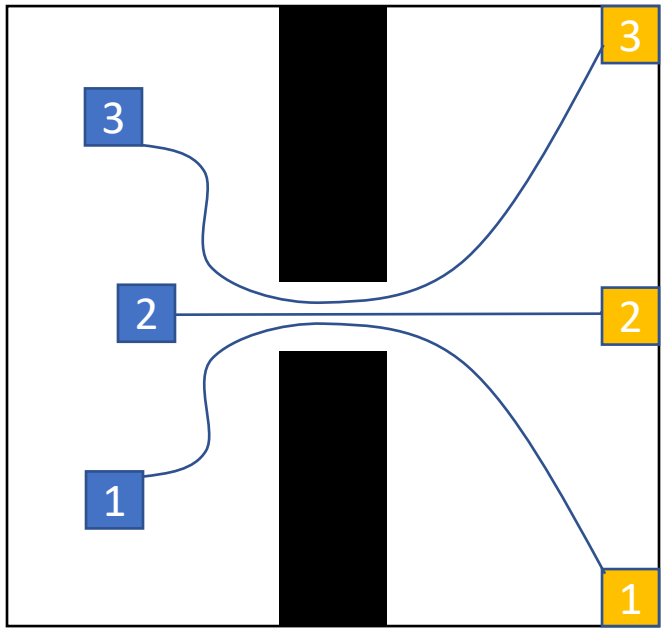
- Decentralized planning:
  - Each robot re-plans its own trajectory
  - Trajectory start time is different
  - We use a finite horizon for collision checking

$$ {}^{i}\Phi(t) \cap \bigcup_{j=0}^{Z-1} [c_{i,0} \oplus c_j(t + \tau_i^s - \tau_j^s)] = \emptyset. $$

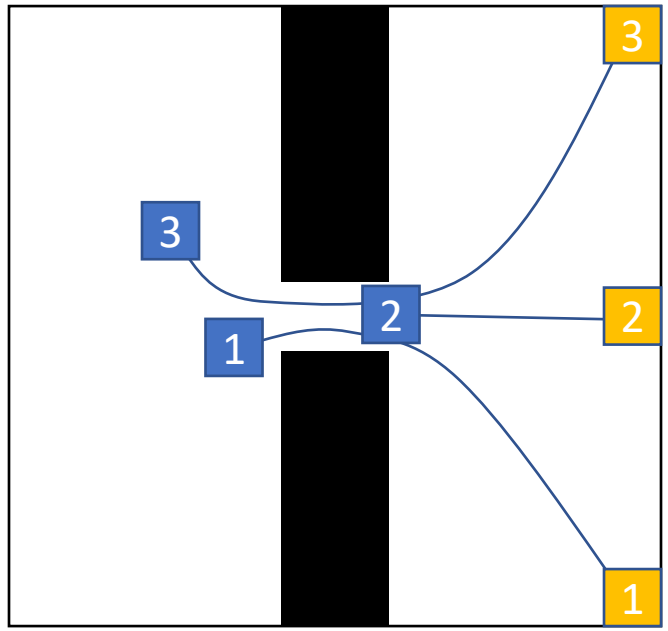
$$ {}^{j}T_c = \min\{\bar{v}_{max}/\bar{a}_{max}, {}^{j}T\}. $$



Re-plan epoch 0



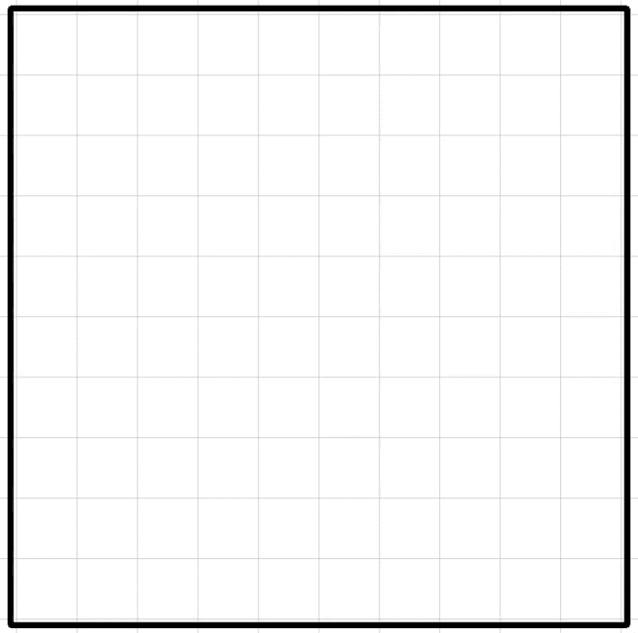
Re-plan epoch 2



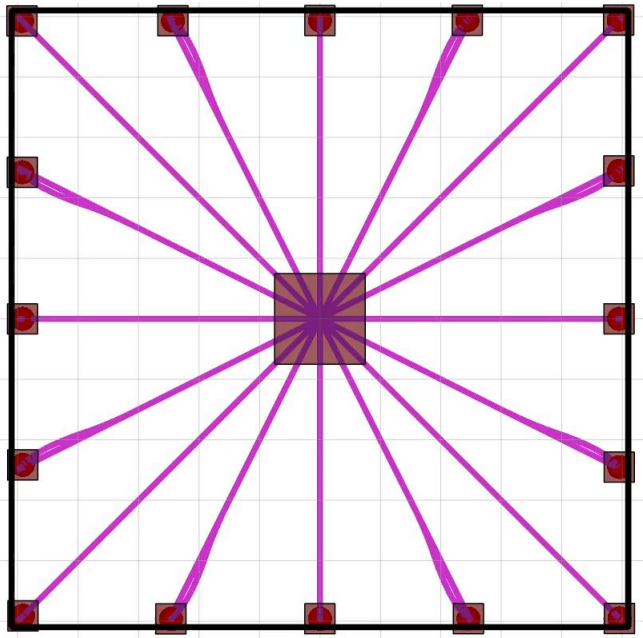
Re-plan epoch 4

Planning in Dynamic Environments III: multi-robot planning

- Decentralized planning:



Tunnel Configuration



Star Configuration

Outline

- Introduction
- Preliminary
- Optimization-based Trajectory Planning
- **Search-based Trajectory Planning**
- Conclusion and Future Work

Outline

- Introduction
- Preliminary
- Optimization-based Trajectory Planning
- Search-based Trajectory Planning
- **Conclusion and Future Work**

Conclusion and Future Work

- Contribution

Method	*Feasibility*	*Safety*	*Optimality*	*Completeness*	*Run time*
Path Planning	Not feasible	Collision free	Globally optimal	Resolution complete	Very fast
Unconstrained QP	May violate constraints	Not guaranteed	Sub-optimal	Not complete	Very fast
MIP	Dynamically feasible	Collision free	Sub-optimal	Complete	Very slow
Short Trajectories	Dynamically feasible	Collision free	Locally optimal	Not complete	Very fast
SFC	Dynamically feasible	Collision free	Sub-optimal	Complete	Fast
SMP	Dynamically feasible	Collision free	Resolution optimal	Globally complete	Fast

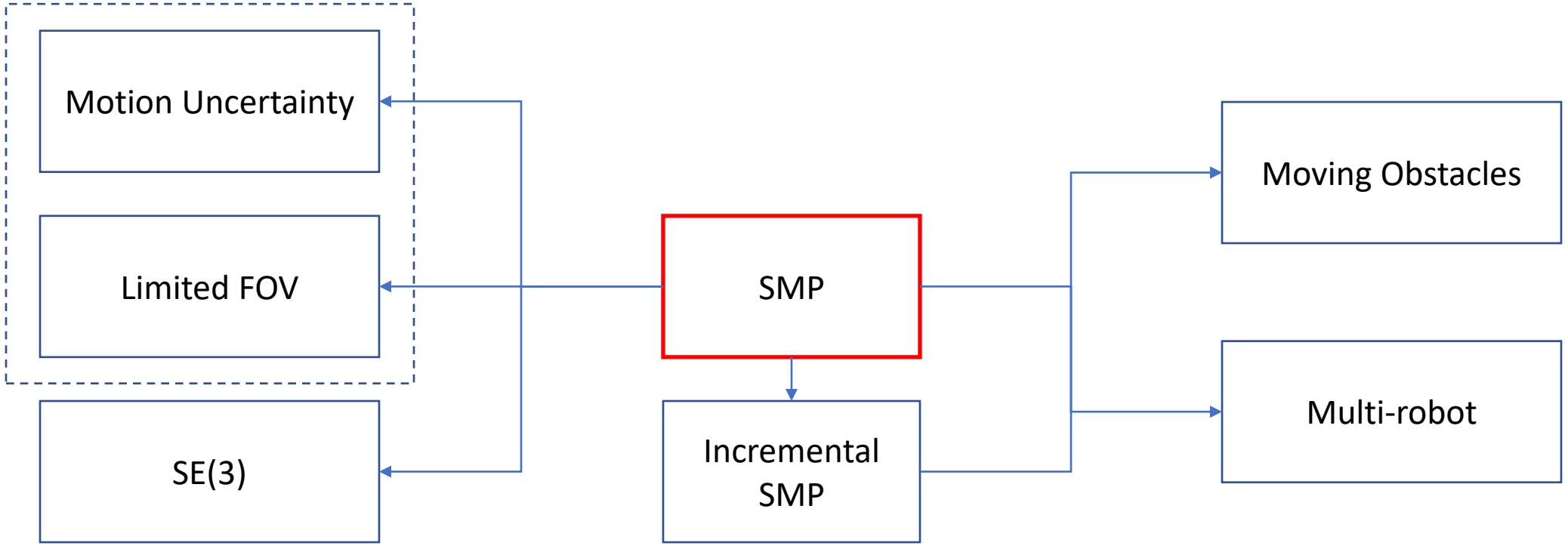
Conclusion and Future Work

- Contribution

	Static Map					Dynamic Map
Method	***Feasibility***	***Safety***	***Optimality***	***Completeness***	***Run time***	
Path Planning	Not feasible	Collision free	Globally optimal	Resolution complete	Very fast	No
Unconstrained QP	May violate constraints	Not guaranteed	Sub-optimal	Not complete	Very fast	
MIP	Dynamically feasible	Collision free	Sub-optimal	Complete	Very slow	
Short Trajectories	Dynamically feasible	Collision free	Locally optimal	Not complete	Very fast	
SFC	Dynamically feasible	Collision free	Sub-optimal	Complete	Fast	
SMP	Dynamically feasible	Collision free	Resolution optimal	Globally complete	Fast	Yes

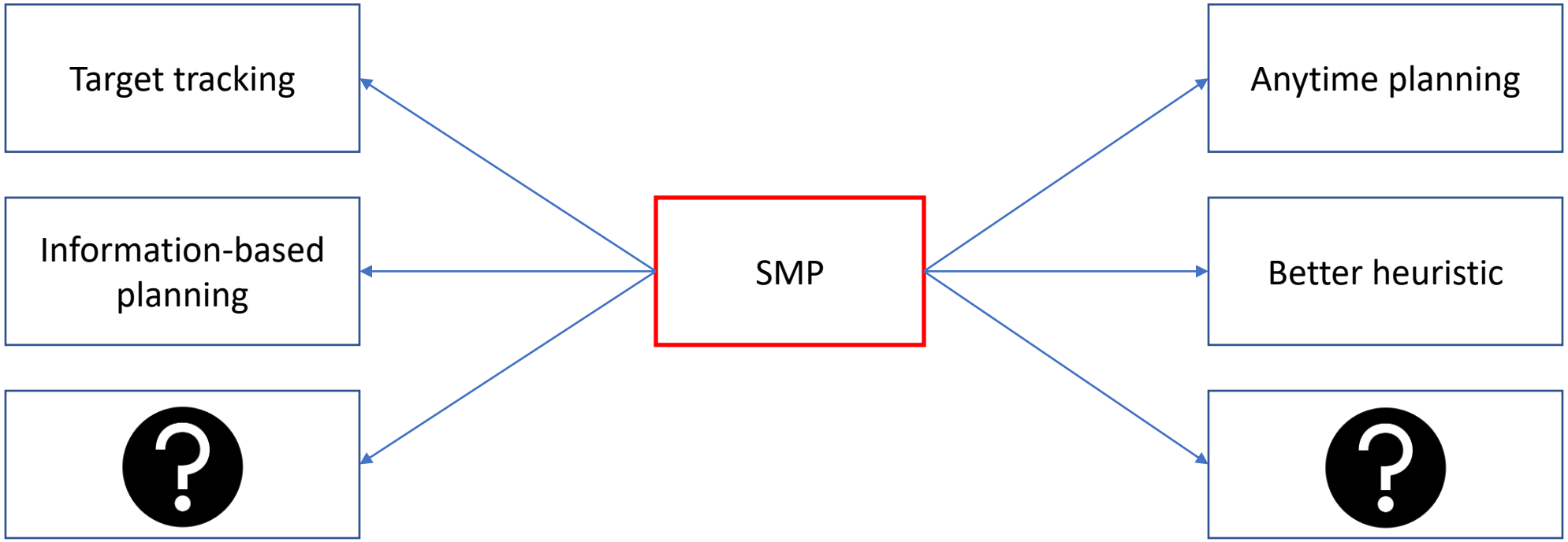
Conclusion and Future Work

- Contribution



Conclusion and Future Work

- Future work



Q & A

- Open-sourced resources:

1. MRSL Quadrotor Simulator: https://github.com/KumarRobotics/mrsl_quadrotor
2. JPS3D: https://github.com/KumarRobotics/jps3d
3. DecompUtil: https://github.com/sikang/DecompUtil
4. DecompROS: https://github.com/sikang/DecompROS
5. Motion Primitive Library: https://github.com/sikang/motion_primitive_library
6. MPL ROS: https://github.com/sikang/mpl_ros

感谢各位聆听

Thanks for Listening

